

EtherCAT[®] Modular Device Profile

Part 3: MDP Fieldbus Gateway Profile Specifications

Document: ETG.5001.3 S (D) V0.1.2

Nomenclature:

ETG-Number	ETG.5001.3
Type	S (Standard)
State	D (Draft)
Version	0.1.2

Created by:	ETG
Contact:	info@ethercat.org
Filename:	ETG5001_3_MDP_Gateways.docx
Date:	05.08.2016

LEGAL NOTICE

Trademarks and Patents

EtherCAT® is a registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany. Other designations used in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owners.

Disclaimer

The documentation has been prepared with care. The technology described is, however, constantly under development. For that reason the documentation is not in every case checked for consistency with performance data, standards or other characteristics. In the event that it contains technical or editorial errors, we retain the right to make alterations at any time and without warning. No claims for the modification of products that have already been supplied may be made on the basis of the data, diagrams and descriptions in this documentation.

Copyright

© EtherCAT Technology Group 08/2011.

The reproduction, distribution and utilization of this document as well as the communication of its contents to others without express authorization is prohibited. Offenders will be held liable for the payment of damages. All rights reserved in the event of the grant of a patent, utility model or design.

CONTENTS

1	Scope	1
1.1	General	1
1.2	Document structure	1
2	References	2
3	Terms, Definitions and Word Usage	3
3.1	Terms and Definitions	3
3.2	Word usage: shall, should, may, can	3
4	Defined Module Profiles	4
Annex A	EtherCAT Master (Module Profile Number 1100)	5
A.1	General	5
A.2	Object Dictionary of the EtherCAT Master	5
A.2.1	Object Areas of the EtherCAT Slaves (Modules)	5
A.2.2	Object Areas of the Device	6
A.3	PDO Mapping and Assign	8
A.4	EtherCAT State Machine	8
A.5	Device configuration	8
A.6	Acyclic Access to the EtherCAT Slaves	8
A.7	Synchronization	8
Annex B	KBus Master (Module Profile Number 1120)	9
B.1	Object Dictionary of the KBus Master	9
B.1.1	Object Areas of the KBus terminals (Modules)	9
B.1.2	Object Areas of the Device	11
B.2	PDO Mapping and Assign	13
B.2.1	Object 0x16nn: RxPDO Mapping of the KBus terminals	13
B.2.2	Object 0x1Ann: TxPDO Mapping of the KBus Slaves	14
B.2.3	Object 0x16FF: RxPDO Mapping of the KBus Control	14
B.2.4	Object 0x1AFF: TxPDO Mapping of the KBus Status	14
B.2.5	Object 0x1C2: RxPDO Assign	14
B.2.6	Object 0x1C13: TxPDO Assign	14
B.3	EtherCAT State Machine	14
B.4	Acyclic Access to the KBus terminals	15
B.5	Synchronization	15
B.5.1	FreeRun Mode	15
B.5.2	Synchron Mode	15
B.5.3	DC Mode	15
Annex C	PROFIBUS DP Master (Module Profile Number 3100)	16
C.1	Object Dictionary of the PROFIBUS DP Master	16
C.1.1	Object Areas of the PROFIBUS DP Slaves (Modules)	16
C.1.2	Object Areas of the Device	22
C.2	PDO Mapping and Assign	28
C.2.1	Objects 0x16nn: RxPDO Mapping of the PROFIBUS DP Slaves	28
C.2.2	Objects 0x1Ann: TxPDO Mapping of the PROFIBUS DP Slaves	29
C.2.3	Object 0x1A81: TxPDO Mapping of PROFIBUS PDO State	29
C.2.4	Object 0x1A82: TxPDO Mapping of PROFIBUS PDO Toggle	30
C.2.5	Objects 0x1A84: TxPDO Mapping of the PROFIBUS DP Slave Status .	30
C.2.6	Object 0x1C12: RxPDO Assign	30
C.2.7	Object 0x1C13: TxPDO Assign	31
C.3	EtherCAT State Machine	31

	C.4	Device configuration	31
	C.5	Acyclic Access to the DP Slaves	32
	C.6	Synchronization	32
	C.6.1	FreeRun Mode	32
	C.6.2	Synchron Mode	32
	C.6.3	DC Mode	32
Annex D		PROFIBUS DP Slave (Module Profile Number 3110)	33
	D.1	Object Dictionary of the PROFIBUS DP Slave Device	33
	D.1.1	Object Areas of the PROFIBUS DP Slave (Module)	33
	D.1.2	Object Areas of the Device	36
	D.2	PDO Mapping and Assign	37
	D.2.1	Object 0x1600: RxPDO Mapping of the PROFIBUS DP Slave	37
	D.2.2	Object 0x1A00: TxPDO Mapping of the PROFIBUS DP Slave	38
	D.2.3	Object 0x1A7F: TxPDO Mapping of PROFIBUS DP Status	38
	D.2.4	Object 0x1C12: RxPDO Assign	38
	D.2.5	Object 0x1C13: TxPDO Assign	39
	D.3	EtherCAT State Machine	39
	D.4	Device configuration	39
	D.5	Synchronization	39
	D.5.1	FreeRun Mode	39
Annex E		Interbus Slave (Module Profile Number 4010)	40
	E.1	General	40
	E.2	Object Dictionary of the Interbus Slave Device	40
	E.2.1	Object Areas of the Interbus Slave (Module)	40
	E.2.2	Object Areas of the Device	41
	E.3	PDO Mapping and Assign	42
	E.3.1	Object 0x1600: RxPDO Mapping of the Interbus Slave	42
	E.3.2	Object 0x1A00: TxPDO Mapping of the Interbus Slave	42
	E.3.3	Object 0x1A7F: Status TxPDO	42
	E.3.4	Object 0x1C12: RxPDO Assign	43
	E.3.5	Object 0x1C13: TxPDO Assign	43
	E.4	EtherCAT State Machine	43
	E.5	Synchronization	43
	E.5.1	FreeRun Mode	43
Annex F		CAN Interface (Module Profile Number 5000)	44
	F.1	General	44
	F.2	Object Dictionary of the CAN Interface	44
	F.2.1	Object Areas of the CAN Interface (Module)	44
	F.2.2	Object Areas of the Device	48
	F.3	PDO Mapping and Assign	50
	F.3.1	Object 0x1600: RxPDO Mapping of the CAN Interface	50
	F.3.2	Object 0x1A00: TxPDO Mapping of the CAN Interface	50
	F.3.3	Object 0x1A85: TxPDO Mapping CAN Status	51
	F.3.4	Object 0x1C12: RxPDO Assign	51
	F.3.5	Object 0x1C13: TxPDO Assign	51
	F.4	EtherCAT State Machine	52
	F.5	Device configuration	52
	F.6	Synchronization	52
	F.6.1	FreeRun Mode	52

	F.6.2	Synchron Mode	52
	F.6.3	DC Mode	52
Annex G		CANopen Master (Module Profile Number 5100)	53
	G.1	General	53
	G.2	Object Dictionary of the CANopen Master.....	53
	G.2.1	Object Areas of the CANopen Slaves (Modules)	53
	G.2.2	Object Areas of the Device.....	62
	G.3	PDO Mapping and Assign	67
	G.3.1	Objects 0x16nn: RxPDO Mapping of the CANopen Slaves.....	67
	G.3.2	Objects 0x1Ann: TxPDO Mapping of CANopen Slaves.....	67
	G.3.3	Object 0x1A81: TxPDO Mapping of CANopen State	68
	G.3.4	Object 0x1A82: TxPDO Mapping of CAN Status	68
	G.3.5	Object 0x1A88...0x1A8F: TxPDO Mapping of PDO Toggle	68
	G.3.6	Object 0x1A83: TxPDO Mapping of CANopen Node State	69
	G.3.7	Object 0x1A84: TxPDO Mapping of CANopen Master Status	69
	G.3.8	Object 0x1A85: TxPDO Mapping of the CANopen Status	70
	G.3.9	Object 0x1C12: RxPDO Assign.....	70
	G.3.10	Object 0x1C13: TxPDO Assign.....	70
	G.4	EtherCAT State Machine	71
	G.5	Device configuration	71
	G.6	Acyclic Access to the CANopen Slaves	71
	G.7	Synchronization	72
	G.7.1	FreeRun Mode	72
	G.7.2	Synchron Mode	72
	G.7.3	DC Mode	72
Annex H		CANopen Slave (Module Profile Number 5110)	73
	H.1	General	73
	H.2	Object Dictionary of the CANopen Slave Device.....	73
	H.2.1	Object Areas of the CANopen Slave (Module).....	73
	H.2.2	Object Areas of the Device.....	78
	H.3	PDO Mapping and Assign	80
	H.3.1	Object 0x1600: RxPDO Mapping of the CANopen Slave	80
	H.3.2	Object 0x1A00: TxPDO Mapping of the CANopen Slave	80
	H.3.3	Object 0x1A83: TxPDO Mapping of the CANopen Status	81
	H.3.4	Object 0x1A85: TxPDO Mapping of the CAN Status	81
	H.3.5	Object 0x1C12: RxPDO Assign.....	81
	H.3.6	Object 0x1C13: TxPDO Assign.....	81
	H.4	EtherCAT State Machine	82
	H.5	Device configuration	82
	H.6	Synchronization	82
	H.6.1	FreeRun Mode	82
Annex I		DeviceNet Master (Module Profile Number 5200)	83
Annex J		DeviceNet Slave (Module Profile Number 5210)	84
Annex K		IO-Link Master (Module Profile Number 6220)	85

TABLES

Table 1: Module Profile Number.....	4
Table 2: EtherCAT Master device Object Dictionary Structure	5
Table 3: EtherCAT Master Objects 0x8nnn: Configuration Data	5
Table 4: EtherCAT Master Objects 0x9nnn: Information Data.....	6
Table 5: EtherCAT Master Objects 0xAxxx: Diagnosis Data.....	6
Table 6: EtherCAT Master Object Areas of the Device	6
Table 7: EtherCAT Master Object 0xF000: Modular Device Profile	7
Table 8: EtherCAT Master Object 0xF002: Detected Modules	7
Table 9: EtherCAT Master Objects 0xF02n: Configured Address List.....	7
Table 10: EtherCAT Master Object 0xF04n: Detected Address List.....	8
Table 11: KBus Master device Object Dictionary Structure	9
Table 12: KBus Master Objects 0x7nn0: Output Data	9
Table 13: KBus Master Objects 0x6nn0: Input Data	10
Table 14: KBus Master Objects 0x9nn0: Information Data	10
Table 15: KBus Master Object Areas of the Device	11
Table 16: KBus Master Object 0xF000: Modular Device Profile	11
Table 17: KBus Master Object 0xF00E: Module PDO group alignment PDO number	11
Table 18: KBus Master Object 0xF00F: Module PDO group alignment	12
Table 19: KBus Master Object 0xF010: Module Profile List.....	12
Table 20: KBus Master Object 0xF030: Detected Module Ident List	12
Table 21: KBus Master Object 0xF040: Detected Address List	12
Table 22: KBus Master Object 0xF050: Detected Module Ident List	13
Table 23: KBus Master Object 0xF100: KBus Status	13
Table 24: KBus Master Object 0xF200: KBus Control	13
Table 25: KBus Master Objects 0x1600-0x16FE: RxPDO Mapping	13
Table 26: KBus Master Objects 0x1A00-0x1AFE: TxPDO Mapping.....	14
Table 27: KBus Master Object 0x16FF: RxPDO Mapping of KBus Control.....	14
Table 28: KBus Master Object 0x1AFF: TxPDO Mapping of KBus Status	14
Table 29: EtherCAT state behavior for the KBus Master fieldbus gateways	15
Table 30: KBus Master Relation of the AoE parameter	15
Table 31: PROFIBUS DP Master Object Dictionary Structure	16
Table 32: PROFIBUS DP Master Objects 0x7nn0: DP Output Data	16
Table 33: PROFIBUS DP Master Objects 0x6nn0: DP Input Data	17
Table 34: PROFIBUS DP Master Objects 0x8nn0: PROFIBUS DP Communication Parameter	18
Table 35: PROFIBUS DP Master Objects 0x8nn1: Prm_Data.....	19
Table 36: PROFIBUS DP Master Objects 0x8nn2: Cfg_Data.....	19
Table 37: PROFIBUS DP Master Objects 0x8nn3: PROFIBUS DPV1 Class 2 Communication Parameter	20
Table 38: PROFIBUS DP Master Objects 0x9nn0: Detected PROFIBUS DP Identification	21
Table 39: PROFIBUS DP Master Objects 0x9nn2: Real_Cfg_Data	21
Table 40: PROFIBUS DP Master Objects 0xAxxx: Diag_Data	22
Table 41: PROFIBUS DP Master Device Object Areas	22
Table 42: PROFIBUS DP Master Object 0xF000: Modular Device Profile	23
Table 43: PROFIBUS DP Master Object 0xF002: Detect Modules Command.....	23
Table 44: PROFIBUS DP Master Object 0xF010: Module Profile List.....	24
Table 45: PROFIBUS DP Master Object 0xF020: Configured Address List	24
Table 46: PROFIBUS DP Master Object 0xF040: Detected Address List	24
Table 47: PROFIBUS DP Master Object 0xF101: DP Master Status	25
Table 48: PROFIBUS DP Master Object 0xF102: DP Slave Status	26
Table 49: PROFIBUS DP Master Object 0xF800: DP Bus Parameter Set.....	27
Table 50: PROFIBUS DP Master Objects 0x1600...0x167C: RxPDO Mapping.....	28
Table 51: PROFIBUS DP Master Objects 0x1A00...0x1A7C: TxPDO Mapping	29
Table 52: PROFIBUS DP Master Objects 0x14nn/0x18nn: PROFIBUS PDO State	29
Table 53: PROFIBUS DP Master Object 0x1A81: TxPDO Mapping of PROFIBUS PDO States	29
Table 54: PROFIBUS DP Master Objects 0x14nn/0x18nn: PROFIBUS PDO Toggle.....	30
Table 55: PROFIBUS DP Master Object 0x1A82: TxPDO Mapping of PROFIBUS PDO Toggle	30
Table 56: PROFIBUS DP Master Objects 0x1A84: TxPDO Mapping of PROFIBUS DP Slave Status	30
Table 57: PROFIBUS DP Master Object 0x1C12: RxPDO Assign	30
Table 58: PROFIBUS DP Master Object 0x1C13: TxPDO Assign	31
Table 59: EtherCAT state behavior for the PROFIBUS DP Master fieldbus gateways	31

Table 60: PROFIBUS DP Master Relation of the AoE parameter	32
Table 61: PROFIBUS DP Slave device Object Dictionary Structure	33
Table 62: PROFIBUS DP Slave Object 0x7000: DP Input Data	33
Table 63: PROFIBUS DP Slave Objects 0x6000: DP Output Data	34
Table 64: PROFIBUS DP Slave Objects 0x8000: PROFIBUS DP Communication Parameter	35
Table 65: PROFIBUS DP Slave Object 0x8002: Real_Cfg_Data	35
Table 66: PROFIBUS DP Slave Device Object Areas	36
Table 67: PROFIBUS DP Slave Object 0xF000: Modular Device Profile	36
Table 68: PROFIBUS DP Slave Object 0xF100: DP Status	36
Table 69: PROFIBUS DP Slave Object 0xF800: DP Bus Parameter Set	37
Table 70: PROFIBUS DP Slave Object 0x1600: RxPDO Mapping	37
Table 71: PROFIBUS DP Slave Object 0x1A00: TxPDO Mapping	38
Table 72: PROFIBUS DP Slave Object 0x1A7F: TxPDO Mapping of PROFIBUS DP Status	38
Table 73: PROFIBUS DP Slave Object 0x1C12RxPDO Assign	38
Table 74: PROFIBUS DP Slave Object 0x1C13TxPDO Assign	39
Table 75: EtherCAT state behavior for the PROFIBUS DP Slave fieldbus gateways	39
Table 76: Interbus Slave device Object Dictionary Structure	40
Table 77: Interbus Slave Object 0x7000: Interbus Send Data	40
Table 78: Interbus Slave Object 0x6000: Interbus Receive Data	40
Table 79: Interbus Slave Object Areas of the Device	41
Table 80: Interbus Slave Object 0xF000: Modular Device Profile	41
Table 81: Interbus Slave Object 0xF100: Interbus Status	41
Table 82: Interbus Slave Object 0xF800: Interbus Parameter Settings	41
Table 83: Interbus Slave Objects 0x1600: RxPDO Mapping	42
Table 84: Interbus Slave Object 0x1A00: TxPDO Mapping	42
Table 85: Interbus Slave Object 0x1A7F: TxPDO Mapping of Interbus Status	42
Table 86: Interbus Slave Object 0x1C12: RxPDO Assign	43
Table 87: Interbus Slave Object 0x1C13: TxPDO Assign	43
Table 88: EtherCAT state behavior for the Interbus Slave fieldbus gateways	43
Table 89: CAN Interface device Object Dictionary Structure	44
Table 90: CAN Interface Object 0x7000: CAN Tx Message Queue	44
Table 91: CAN Interface Object 0x7001: Exented CAN TX Message Queue	45
Table 92: CAN Interface Object 0x6000: Standard Rx Message Queue	46
Table 93: CAN Interface Object 0x6001: Extended CAN Rx Message Queue	47
Table 94: CAN Interface Object 0x8000: CAN Interface Confuiuration	47
Table 95: CAN Interface Object 0x8001: Rx Filter Table	48
Table 96: CAN Interface device Object Areas	48
Table 97: CAN Interface Object 0xF000: Modular Device Profile	48
Table 98: CAN Interface Object 0xF108: CAN Status	49
Table 99: CAN Interface Object 0xF800: CAN Bus Parameter Set	49
Table 100: CAN Interface Object 0x1600: RxPDO Mapping (if Standard CAN msg queue selected) .	50
Table 101: CAN Interface Object 0x1600: RxPDO Mapping (if Extended CAN msg queue selected) .	50
Table 102: CAN Interface Object 0x1A00: TxPDO Mapping (if Standard CAN msg queue selected) .	50
Table 103: CAN Interface Object 0x1600: TxPDO Mapping (if Extended CAN msg queue selected) .	50
Table 104: CAN Interface Object 0x1A85: TxPDO Mapping of CAN Status	51
Table 105: CAN Interface Object 0x1C12: RxPDO Assign	51
Table 106: CAN Interface Object 0x1C13: TxPDO Assign	51
Table 107: CAN interface EtherCAT state behavior	52
Table 108: CANopen Master device Object Dictionary Structure	53
Table 109: CANopen Master Object 0x7nn0...0x7nn1: Output Data	53
Table 110: CANopen Master Object 0x6nn0...0x6nn1: Input Data	54
Table 111: CANopen Master Objects 0x6nn4...0x6nn5: TxPDO Toggle	55
Table 112: CANopen Master Object 0x8nn0: CAN Communication Parameter	56
Table 113: CANopen Master Object 0x8nn3: SDO Init Cmds	57
Table 114: CANopen Master Object 0x8nn6...0x8nn7: TxPDO Configuration	57
Table 115: CANopen Master Object 0x8nn8...0x8nn9: RxPDO Configuration	58
Table 116: CANopen Master Object 0x9nn0: Detected CANopen Identification	59
Table 117: CANopen Master Object 0x9nn6...0x9nn7: Detected CAN TxPDO Configuration	59
Table 118: CANopen Master Object 0x9nn8...0x9nn9: Detected CAN RxPDO Configuration	60
Table 119: CANopen Master Object 0x9nnA...0x9nnB: Detected CAN TxPDO Mapping	61
Table 120: CANopen Master Object 0x9nnC...0x9nnD: Detected RxPDO Mapping	61
Table 121: CANopen Master Device Object Areas	62

Table 122: CANopen Master Object 0xF000: Modular Device Profile	62
Table 123: CANopen Master Object 0xF002: Detect Modules Command	63
Table 124: CANopen Master Object 0xF010: Module Profile List.....	63
Table 125: CANopen Master Object 0xF020: Configured Address List.....	64
Table 126: CANopen Master Object 0xF040: Detected Address List	64
Table 127: CANopen Master Object 0xF101: CANopen Master Status	64
Table 128: CANopen Master Object 0xF102: Node state list	65
Table 129: CANopen Master Object 0xF108: CAN Status	66
Table 130: CANopen Master Object 0xF800: CAN Bus Parameter Set	66
Table 131: CANopen Master Objects 0x1600...0x167E: RxPDO Mapping.....	67
Table 132: CANopen Master Objects 0x1A00...0x1A7E: TxPDO Mapping	67
Table 133: CANopen State.....	68
Table 134: CANopen Master Object 0x1A81: TxPDO Mapping of CANopen States	68
Table 135: CANopen Master Object 0x1A82: TxPDO Mapping of CAN Status	68
Table 136: CANopen Master Objects 0x1A88...0x1A8F: TxPDO Mapping of CANopen TxPDO Toggle	69
Table 137: CANopen Master object 0x1A83: TxPDO Mapping for Node States	69
Table 138: CANopen Master object 0x1A84: TxPDO Mapping for CANopen Master Status	69
Table 139: CANopen Master object 0x1A85: TxPDO Mapping for CAN Status	70
Table 140: CANopen Master Object 0x1C12: RxPDO Assign	70
Table 141: CANopen Master Object 0x1C13: TxPDO Assign	70
Table 142: CANopen Master EtherCAT state behavior	71
Table 143: CANopen Master Relation of AoE parameter	72
Table 144: CANopen Slave device Object Dictionary Structure	73
Table 145: CANopen Slave Objects 0x7nn0...0x7nn1: Output Data.....	73
Table 146: CANopen Slave Objects 0x6nn0...0x6nn1: Input Data.....	74
Table 147: CANopen Slave Object 0x8000: CAN Communication Parameter.....	75
Table 148: CANopen Slave Object 0x8006...0x8007: TxPDO Configuration.....	76
Table 149: CANopen Slave Object 0x8008...0x8009: RxPDO Configuration	77
Table 150: CANopen Slave Object 0x800A-0x800B: TxPDO Mapping	77
Table 151: CANopen Slave Object 0x800C-0x800D: RxPDO Mapping	78
Table 152: CANopen Slave Device Object Areas	78
Table 153: CANopen Slave Object 0xF000: Modular Device Profile	79
Table 154: CANopen Slave Object 0xF100: CANopen Status	79
Table 155: CANopen Slave Object 0xF108: CAN Status	79
Table 156: CANopen Slave Object 0xF800: CAN Bus Parameter Set	80
Table 157: CANopen Slave Object 0x1600: RxPDO Mapping	80
Table 158: CANopen Slave Object 0x1A00: TxPDO Mapping	80
Table 159: CANopen Slave Object 0x1A83: TxPDO Mapping of CANopen Status	81
Table 160: CANopen Slave Object 0x1A85: TxPDO Mapping of CAN Status	81
Table 161: CANopen Slave Object 0x1C12: RxPDO Assign	81
Table 162: CANopen Slave Object 0x1C13: TxPDO Assign	81
Table 163: CANopen slave EtherCAT state behavior	82

FIGURES

Figure 1: Example of MDP devices	1
--	---

ABBREVIATIONS

μC	Microcontroller
ACK	Acknowledgement
ADS	Automation Device Specification
AoE	ADS over EtherCAT
C	Conditional
CAN	Controller Area Network
CMD	Command
COB-ID	Communication Object Identifier (CANopen)
CoE	CAN Application Protocol over EtherCAT
COS	Change of State
DC	Distributed Clock
DPRAM	Dual-Ported RAM
ECAT	EtherCAT
ENI	EtherCAT Network Information (EtherCAT XML Master Configuration)
ENUM	Enumeration
EoE	Ethernet over EtherCAT
ESC	EtherCAT Slave Controller
ESI	EtherCAT Slave Information (EtherCAT XML Devices Description)
ESM	EtherCAT State Machine
ETG	EtherCAT Technology Group
FMMU	Fieldbus Memory Management Unit
FoE	File Access over EtherCAT
FPMR	Configured Address Physical Read Multiple Write
FPRD	Configured Address Physical Read
FPRW	Configured Address Physical ReadWrite
FPWR	Configured Address Physical Write
I/O	Input/Output
IDN	Identification Number (Servo Profile Identifier)
IEC	International Electrotechnical Commission
INT	Integer
IRQ	Interrupt Request
LRD	Logical Read
LRW	Logical ReadWrite
LSB	Least Significant Bit
LWR	Logical Write
M	Mandatory
MAC	Media Access Controller
MI	(PHY) Management Interface
MII	Media Independent Interface
ms	milliseconds (10 ⁻³ seconds)
MSB	Most Significant Bit
NIC	Network Interface Card
NOP	No Operation
ns	nanoseconds (10 ⁻⁹ seconds)
O	Optional
OD	Object Dictionary
OP	Operational
OS	Oversampling
PDO	Process Data Object
PREOP	Pre-Operational
RAM	Random-Access Memory
RD	Read
RO	Read-Only
ROM	Read-Only Memory
RW	Read-Write
RX	Reveice
SAFEOP	Safe-Operational
SDO	Service Data Object
SI	Subindex
SM	Sync Manager

SoE	Servo Profile over EtherCAT
SOF	Start of Frame
SPI	Serial Peripheral Interface
SU	Sync Unit
TX	Transmit
WD	Watchdog
WKC	Working Counter
WR	Write
XML	eXtensible Markup Language

1 Scope

1.1 General

The Modular Device Profile (MDP) defines a modeling of structures within in a device. Mainly the object dictionary structure and corresponding behaviour of the entries is defined by the MDP. The intention is to provide an easy way for master and configuration tools to handle the devices.

A modular structure can be used for all kind of devices that supports physical or logical modules. This can be for example:

- Gateways from Fieldbusses to EtherCAT
- Extendable bus coupler with an internal backbone (e.g. sliced I/Os)
- Multi Axis servo drives (physical modules) with each axis having independent operation modes (logic modules)

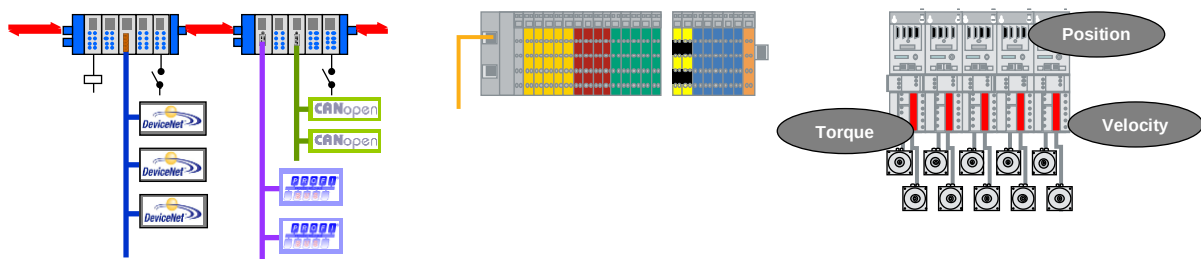


Figure 1: Example of MDP devices

The Device Description for Devices according to the MDP Profile is defined in [7].

1.2 Document structure

The MDP specification is divided in the following subparts:

- ETG.5001.1: General MDP Device Model
- ETG.5001.2: MDP Module Device Specification
- ETG.5001.3: MDP Fieldbus Gateway Profile Specification
- ETG.5001.4: MDP Safety Module Specification

2 References

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ETG Standards

- [1] ETG.1000.2: Physical Layer service definition and protocol specification
- [2] ETG.1000.3: Data Link Layer service definition
- [3] ETG.1000.4: Data Link Layer protocol specification
- [4] ETG.1000.5: Application Layer service definition
- [5] ETG.1000.6: Application Layer protocol specification
- [6] ETG.1020: EtherCAT Protocol Enhancements
- [7] ETG.2000: EtherCAT Slave Information
- [8] ETG.2100: EtherCAT Network Information
- [9] ETG.5001.1: Modular Device Profile – Part 1: General MDP Device Model
- [10] ETG.5001.2: Modular Device Profile – Part 2: MDP Module Device Specification
- [11] ETG.5001.3: Modular Device Profile – Part 3: MDP Fieldbus Gateway Profile Specification
- [12] ETG.5001.4: Modular Device Profile – Part 4: MDP Safety Module Specification

Other References

- [13] IEC 61158-x-12 (all parts for type 12): Industrial communication networks – Fieldbus specifications
- [14] IEC 61784-2: Industrial communication networks – Profiles – Part 2: Additional fieldbus profiles for real-time networks based on ISO/IEC 8802-3
- [15] IEC 61800-7: Adjustable speed electrical power drives systems – Part 7: Generic interface and use of profiles for power drive systems

3 Terms, Definitions and Word Usage

3.1 Terms and Definitions

The terms and definitions of ETG.1000 series shall be fully valid, unless otherwise stated.

3.2 Word usage: shall, should, may, can

The word *shall* is used to indicate mandatory requirements strictly to be followed in order to conform to the standard and from which no deviation is permitted (*shall* equals *is required to*).

The word *should* is used to indicate that among several possibilities one is recommended as particularly suitable, without mentioning or excluding others; or that a certain course of action is preferred but not necessarily required; or that (in the negative form) a certain course of action is deprecated but not prohibited (*should* equals *is recommended that*).

The word *may* is used to indicate a course of action permissible within the limits of the standard (*may* equals *is permitted to*).

The word *can* is used for statements of possibility and capability, whether material, physical, or causal (*can* equals *is able to*).

4 Defined Module Profiles

Table 1 lists the standardized Module Profiles in this part of the standard. A complete overview of all standardized Module Profiles can be found in [9].

Table 1: Module Profile Number

Module Profile Number	Name	see
1100	EtherCAT Master	Annex A
1120	KBus Master	Annex B
3100	PROFIBUS Master	Annex C
3110	PROFIBUS Slave	Annex D
4010	Interbus Slave	Annex E
5000	CAN Interface	Annex F
5100	CANopen Master	Annex G
5110	CANopen Slave	Annex H
5200	DeviceNet Master	Annex I
5210	DeviceNet Slave	Annex J
6220	IO Link Master	Annex K

Annex A EtherCAT Master (Module Profile Number 1100)

A.1 General

The EtherCAT Master object dictionary is based on the modular device profile (fieldbus gateway) with one object per module in each object area. So up to 4080 (not 4096 because of objects 0xF02n and 0xF04n) modules (EtherCAT slaves) can be supported.

A.2 Object Dictionary of the EtherCAT Master

The object dictionary of the device contains the following object areas as shown in Table 2.

Table 2: EtherCAT Master device Object Dictionary Structure

Index	Object Dictionary Area
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x8000...0x8FFF	Configuration Area (Expected configuration of EtherCAT slaves)
0x9000...0x9FFF	Information Area (Detected configuration of EtherCAT slaves)
0xA000...0xAFFF	Diagnosis Area (Diagnosis of EtherCAT slaves)
0xF000...0xFFFF	Device Area

A.2.1 Object Areas of the EtherCAT Slaves (Modules)

A.2.1.1 Configuration Data

The Configuration Data contains the information about the configured EtherCAT slaves that for example a tool can use to communicate via the mailbox gateway.

The Configuration Data objects 0x8nnn as shown in Table 3 are mandatory for an EtherCAT Master.

Table 3: EtherCAT Master Objects 0x8nnn: Configuration Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x8000	1	Fixed Station Address	UINT16	RO	Fixed Station Address of the first EtherCAT Slave (same value as 0xF020:01)
	2	Type	VISIBLE-STRING	RO	Type of the first EtherCAT Slave configured
	3	Name	VISIBLE-STRING	RO	Name of the first EtherCAT Slave configured (object 0x1008 of the EtherCAT slave)
	4	Device Type	UINT32	RO	Device Type of the first EtherCAT Slave configured (object 0x1000 of the EtherCAT slave)
	5	Vendor Id	UINT32	RO	Vendor ID of the first EtherCAT Slave configured (Identy entry 0x1018:01 of the EtherCAT slave)
	6	Product Code	UINT32	RO	Product Code of the first EtherCAT Slave configured (Identy entry 0x1018:02 of the EtherCAT slave)
	7	Revision Number	UINT32	RO	Revision Number of the first EtherCAT Slave configured (Identy entry 0x1018:03 of the EtherCAT slave)
	8	Serial Number	UINT32	RO	Serial Number of the first EtherCAT Slave configured (Identy entry 0x1018:04 of the EtherCAT slave)
	33	Mailbox Out Size	UINT16	RO	Mailbox Write Size (SM0) of the first EtherCAT Slave configured
	34	Mailbox In Size	UINT16	RO	Mailbox Read Size (SM1) of the first EtherCAT Slave configured
	35	Link Status	BYTE	RO	Link Status of the first EtherCAT Slave configured
..					
0x8001	1	Fixed Station Address	UINT16	RO	Fixed Station Address of the second EtherCAT Slave (same value as 0xF020:02)
...		...			

A.2.1.2 Information Data

The Information Data as shown in Table 4 contains the information about the connected EtherCAT slaves. It will be available when the Scan Boxes Command is executed or it will be updated automatically by the EtherCAT Master.

The Information Data is optional for an EtherCAT Master.

Table 4: EtherCAT Master Objects 0x9nnn. Information Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x9000	1	Fixed Station Address	UINT16	RO	Fixed Station Address of the first EtherCAT Slave found (same value as 0xF040:01)
	5	Vendor Id	UINT32	RO	Vendor ID of the first EtherCAT Slave found (entry 0x1018:01 of the EtherCAT slave)
	6	Product Code	UINT32	RO	Product Code of the first EtherCAT Slave found (entry 0x1018:02 of the EtherCAT slave)
	7	Revision Number	UINT32	RO	Revision of the first EtherCAT Slave found (entry 0x1018:03 of the EtherCAT slave)
	8	Serial Number	UINT32	RO	Serial Number of the first EtherCAT Slave found (entry 0x1018:04 of the EtherCAT slave)
	32	DL Status Register	UINT16	RO	DL Status (Register 0x110-0x111) of the first EtherCAT found
0x9001	1	Fixed Station Address	UINT16	RO	Fixed Station Address of the second EtherCAT Slave found (same value as 0xF040:02)
...		...			

A.2.1.3 Diagnosis Data

The Diagnosis Data contains the status and diagnosis information about the EtherCAT slaves or the connection to the EtherCAT slaves.

The Diagnosis Data objects 0xAAnnn as shown in Table 5 are optional for an EtherCAT Master.

Table 5: EtherCAT Master Objects 0xAAnnn: Diagnosis Data

Index	Subindex	Name	Data Type	Access	Description / Value
0xA000	1	AL Status	UINT16	RO	AL Status (Register 0x130-0x131) of the first EtherCAT slave configured
	2	AL Control	UINT16	RW	AL Control (Register 0x120-0x121) of the first EtherCAT slave configured
	...				
0xA001	1	AL Status	UINT16	RO	AL Status (Register 0x130-0x131) of the second EtherCAT slave configured
	...				

A.2.2 Object Areas of the Device

The objects of the Device Area shown in Table 6 are standardized for an EtherCAT Master. All other objects in the area 0xF000-0xFFFF are reserved for profile extensions.

Table 6: EtherCAT Master Object Areas of the Device

Index	Object Name
0xF000	Modular device profile
0xF002	Detect modules
0xF020	Configured address list
0xF040	Detected address list

A.2.2.1 Object 0xF000: Modular Device Profile

The Modular device profile object (0xF000) shown in Table 7 for an EtherCAT Master.

Table 7: EtherCAT Master Object 0xF000: Modular Device Profile

Index	Subindex	Name	Data Type	Access	Description / Value
0xF000	1	Module Index Distance	UINT16	RO	Index distance between two modules = 0x01
	2	Maximum Number of Modules	UINT16	RO	Maximum number of EtherCAT Slaves connected to the EtherCAT Master = 4080
	3	General configuration	UINT32	RO	Available entries in objects 0x8nn0 = 0x000000FF
	4	General information	UINT32	RO	Available entries in objects 0x9nn0 = 0x000000F1 (if information data supported) or 0x00000000 (if information data not supported)

A.2.2.2 Object 0xF002: Detect Modules Command

The Detect Modules Command object (0xF002) shown in Table 8 is optional for an EtherCAT Master.

Table 8: EtherCAT Master Object 0xF002: Detected Modules

Index	Subindex	Name	Data Type	Access	Description / Value
0xF002	1	Scan Command Request	OCTET-STRING[2]	RW	Command Request Byte 0,1: 0 When this subindex is written, the EtherCAT Master shall scan the EtherCAT bus and update the objects 0xF04x and 0x9nnn
	2	Scan Command Status	UINT8	RO	Command Status 1: Command is finished, no error 3: Command is finished, error 100-199: 0-99% of the command is done 255: Command is executing
	3	Scan Command Response	OCTET-STRING[6]	RO	Byte 0: Same as Subindex 2 Command Response Byte 1: Always 0 Byte 2...3: 0: No error > 0: Vendor specific error code Byte 4...5: Number of EtherCAT slaves found

A.2.2.3 Objects 0xF02n: Configured Address List

The Configured address list objects (0xF02n) shown in Table 9 is mandatory for an EtherCAT Master.

Table 9: EtherCAT Master Objects 0xF02n: Configured Address List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF020	1	Subindex 001	UINT16	RO	Fixed Station Address of the first EtherCAT Slave configured
	2	Subindex 002	UINT16	RO	Fixed Station Address of the second EtherCAT Slave configured
...					
	255	Subindex 255	UINT16	RO	Fixed Station Address of the 255th EtherCAT Slave configured
0xF021	1	Subindex 001	UINT16	RO	Fixed Station Address of the 256th EtherCAT Slave configured
...					
0xF02F	255	Subindex 255	UINT16	RO	Fixed Station Address of the 4080th EtherCAT Slave configured

A.2.2.4 Objects 0xF04n: Detected Address List

The Detected address list object (0xF04n) shown in Table 10 is optional for an EtherCAT Master.

Table 10: EtherCAT Master Object 0xF04n: Detected Address List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF040	1	Subindex 001	UINT16	RO	Fixed Station Address of the first EtherCAT Slave found
	2	Subindex 002	UINT16	RO	Fixed Station Address of the second EtherCAT Slave found
...					
	255	Subindex 255	UINT16	RO	Fixed Station Address of the 255th EtherCAT Slave found
0xF041	1	Subindex 001	UINT16	RO	Fixed Station Address of the 256th EtherCAT Slave found
...					
0xF04F	255	Subindex 255	UINT16	RO	Fixed Station Address of the 4080th EtherCAT Slave found

A.3 PDO Mapping and Assign

The PDO Mapping and Assign of the EtherCAT master is not standardized because the EtherCAT Master Gateway object dictionary only supports acyclic communication.

A.4 EtherCAT State Machine

The EtherCAT State Machine of the EtherCAT master is not standardized because the EtherCAT Master Gateway only supports acyclic communication.

A.5 Device configuration

The device configuration of the EtherCAT master is standardized by the master's configuration, see [8].

A.6 Acyclic Access to the EtherCAT Slaves

The acyclic access to the EtherCAT slaves shall be supported by using the mailbox gateway.

A.7 Synchronization

The synchronization modes of the EtherCAT master are not standardized because the mailbox gateway only supports acyclic communication.

Annex B KBus Master (Module Profile Number 1120)

The KBus Master gateway is based on the modular device profile (modular device). The KBus Master gateway supports up to 255 modules (KBus terminals).

B.1 Object Dictionary of the KBus Master

The object dictionary of the device contains the object areas described in Table 11.

Table 11: KBus Master device Object Dictionary Structure

Index	Object Name
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x6000...0x6FFF	Input Area (Input Data of the KBus terminals)
0x7000...0x7FFF	Output Area (Output Data of the KBus terminals)
0x9000...0x9FFF	Information Area (Detected KBus terminals)
0xF000...0xFFFF	Device Area

B.1.1 Object Areas of the KBus terminals (Modules)

B.1.1.1 Output Data

The Output Data of the KBus master represent the Output Data of the KBus terminals. Each entry of these objects is related to a variable of a KBus terminal (see Table 12). The Output Data is mandatory for a KBus Master if KBus terminals with Output Data are connected.

Table 12: KBus Master Objects 0x7nn0: Output Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x7000	1	1st Output Data variable of the 1st terminal	depend on variable	RO	Represents the Output Data of the first variable of the first KBus terminal
	2	2nd Output Data variable of the 1st terminal	depend on variable	RO	Represents the Output Data of the second variable of the first KBus terminal
..					
0x7010	1	1st Output Data variable of the 2nd terminal	depend on variable	RO	Represents the Output Data of the first variable of the second KBus terminal
...		...			

When a KBus terminal does not have Output Data, the related object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The objects 0x7nn8...0x7nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x7nn1...0x7nn7 are reserved for profile extensions.

B.1.1.2 Input Data

The Input Data of the KBus master represent the Input Data of the KBus terminals. Each entry of these objects is related to a variable of a KBus terminal (see Table 13). The Input Data is mandatory for a KBus Master if KBus terminals with Input Data are connected.

Table 13: KBus Master Objects 0x6nn0: Input Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x6000	1	1st Input Data variable of the 1st terminal	depend on variable	RO	Represents the Input Data of the first variable of the first KBus terminal
	2	2nd Input Data variable of the 1st terminal	depend on variable	RO	Represents the Input Data of the second variable of the first KBus terminal
...					
0x6010	1	1st Input Data variable of the 2nd terminal	depend on variable	RO	Represents the Input Data of the first variable of the second KBus terminal
...					

When a KBus terminal does not have Input Data, the related object does not exist and a SDO Upload or SDO Information service with the related index addressed shall return the abort code 0x06020000 (object doesn't exist).

The objects 0x6nn8...0x6nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x6nn1...0x6nn7 are reserved for profile extensions.

B.1.1.3 Information Data

The Information Data contain the information about the connected terminals as shown in Table 14. It will be available after power-on. The Information Data is mandatory for a KBus Master.

Table 14: KBus Master Objects 0x9nn0: Information Data

Index	Subindex	Name	Data Type	Access	Description / Value			
0x9000	9	Module PDO Group, 1st KBus terminal	UINT16	RO	Digital terminals shall have Module PDO Group 2, non-digital terminals shall have the Module PDO Group 1 because the non-digital terminals will be assigned before all digital terminals in the process data			
	10	Module Ident, 1st KBus terminal	UINT32	RO	Module Ident corresponds to the module ident of the terminal in the device description For Bus Couplers and intelligent (e.g. analog) terminals the terminal identification is shown as plain text. For non-intelligent (digital) terminals, the following coding is essential:			
					Bit	Value	Meaning	
					15	0 _{bin}	intelligent (e.g. analog) terminal	
						1 _{bin}	non-intelligent (digital) terminal	
					14...8	xxxxxx _{bin}	Size of terminal in bit (range: 0 to 63 bit)	
					7	1 _{bin}	Shift register extension for control panels	
					6...3	0 _{bin}	reserved	
					2	1 _{bin}		
					1	1 _{bin}	Output terminal	Special case: 00 _{bin} means digital terminal with input and output data
					0	1 _{bin}	Input terminal	
...								
0x9010	9	Module PDO Group, 2nd KBus terminal	UINT16	RO	Digital terminals shall have Module PDO Group 2, non-digital terminals shall have the Module PDO Group 1 because the non-digital terminals will be assigned before all digital terminals in the process data			
...								

The objects 0x9nn8...0x9nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x9nn1...0x9nn7 are reserved for profile extensions.

B.1.2 Object Areas of the Device

The following objects in Table 15 of the Device Area are standardized for a KBus Master. All other objects in the area 0xF100...0xFFFF are vendor specific but shall be used according to the Modular Device Profile.

Table 15: KBus Master Object Areas of the Device

Index	Object Dictionary Area
0xF000	Modular device profile
0xF00E	Module PDO group alignment PDO number
0xF00F	Module PDO group alignment
0xF010	Module Profile List
0xF030	Configured Module List
0xF040	Detected Address List
0xF050	Detected module ident list (see 0x9nn0:0A)
0xF100	KBus Status
0xF200	KBus Control

B.1.2.1 Object 0xF000: Modular Device Profile

The Modular device profile object (0xF000) is mandatory for a KBus Master and described in Table 16.

Table 16: KBus Master Object 0xF000: Modular Device Profile

Index	Subindex	Name	Data Type	Access	Description / Value
0xF000	1	Index distance	UINT16	RO	Index distance between two modules = 0x10
	2	Maximum number of modules	UINT16	RO	Maximum number of KBus terminals connected to the KBus Master = 255
	3	General configuration	UINT32	RO	Available entries in objects 0x8nn0 = 0x00000000
	4	General information	UINT32	RO	Available entries in objects 0x9nn0 = 0x00000300
	5	Module PDO Group Device	UINT16	RO	Module PDO Group of the Device = 0, because the Control and Status Data will assigned before all terminals in the process data

B.1.2.2 Object 0xF00E: Module PDO group alignment PDO number

The Module PDO group alignment PDO number object described in Table 17 contains the PDOs used for the mapping alignment of the module groups.

Table 17: KBus Master Object 0xF00E: Module PDO group alignment PDO number

Index	Subindex	Name	Data Type	Access	Description / Value
0xF00E	1	Subindex001	UINT16	RO	0 No Align is needed after Module PDO Group 0 (Status or Control Data of the device)
	2	Subindex002	UINT16	RO	0x100 PDO 257 is used for the align after Module PDO Group 1 (non-digital terminals)
	3	Subindex003	UINT16	RO	0x101 PDO 258 is used for the align after Module PDO Group 2 (digital terminals)

B.1.2.3 Object 0xF00F: Module PDO group alignment

The Module PDO group alignment object described in Table 18 contains the alignment information for every Module PDO Group.

Table 18: KBus Master Object 0xF00F: Module PDO group alignment

Index	Subindex	Name	Data Type	Access	Description / Value
0xF00F	1	Alignement PDO Group 0	UINT16	RO	0 - No Align is needed after Module PDO Group 0 (Status or Control Data of the device, because the size of the Status Data and Control Data is fixed and already aligned)
	2	Alignement PDO Group 1	UINT16	RO	2 A Word Align is needed after Module PDO Group 1 (non-digital terminals) that the process data of the digital terminals is always located on an even byte address
	3	Alignement PDO Group 2	UINT16	RO	2 A Word Align is needed after Module PDO Group 2 (digital terminals) that the process data size is always even

B.1.2.4 Object 0xF010: Module Profile List

The Module Profile List contains the module profile numbers of the modules.

Table 19: KBus Master Object 0xF010: Module Profile List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF010	1	Module Profile 1 st Terminal	UINT32	RO	Module Profile of the first terminal
	2	Module Profile 2 nd Terminal	UINT32	RO	Module Profile of the second terminal
	255	Module Profile 255 th Terminal	UINT32	RO	Module Profile of the 255 th terminal

B.1.2.5 Object 0xF030: Configured Module Ident List

The Configured Module Ident List contains the module Ident numbers of the configured modules.

Table 20: KBus Master Object 0xF030: Detected Module Ident List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF030	1	Module Ident 1 st Terminal	UINT32	RO	Module Ident of the first terminal (same value as 0x8000:0A)
	2	Module Ident 2 nd Terminal	UINT32	RO	Module Ident of the second terminal (same value as 0x8010:0A)
	255	Module Ident 255 th Terminal	UINT32	RO	Module Ident of the 255 th terminal (same value as 0x8FE0:0A)

The EtherCAT Master should download the Configured Module Ident List so that the EtherCAT Slave can compare the expected module configuration.

B.1.2.6 Object 0xF040: Detected Address List

The Detected Address List contains the addresses of the detected modules.

Table 21: KBus Master Object 0xF040: Detected Address List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF040	1	Module Address 1 st Terminal	UINT32	RO	Address of the first terminal (number of terminal)
	2	Module Address 2 nd Terminal	UINT32	RO	Address of the second terminal
	255	Module Address 255 th Terminal	UINT32	RO	Address of the 255 th terminal

B.1.2.7 Object 0xF050: Detected Module Ident List

The Detected Module Ident List contains the module ident numbers of the detected modules.

Table 22: KBus Master Object 0xF050: Detected Module Ident List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF050	1	Module Ident 1 st Terminal	UINT32	RO	Module Ident of the first terminal (same value as 0x9000:0A)
	2	Module Ident 2 nd Terminal	UINT32	RO	Module Ident of the second terminal (same value as 0x9010:0A)
	255	Module Ident 255 th Terminal	UINT32	RO	Module Ident of the 255 th terminal (same value as 0x9FE0:0A)

B.1.2.8 Object 0xF100: Device Status

The Device status (see Table 23) is TxPDO mappable and contains the status of the KBus.

Table 23: KBus Master Object 0xF100: KBus Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF100	1	KBus Status	UINT16	RO	Bit 0 : Kbus Error Bit 1...4: reserved, shall be Zero Bit 5 : Overrun -> EtherCAT cycle shorter than K-Bus cycle Bit 6 : No valid inputs Bit 7 : No valid outputs Bit 8...13: reserved, shall be Zero Bit 14: Input Toggle - will be toggled every time when new Input Data were received from the KBus Bit 15: Input State - will be set in case of a KBus error

B.1.2.9 Object 0xF200: Device Control

The Device control (see Table 24) is RxPDO mappable and contains the control of the KBus.

Table 24: KBus Master Object 0xF200: KBus Control

Index	Subindex	Name	Data Type	Access	Description / Value
0xF200	1	KBus Control	UINT16	RW	Bit 4 : disable Outputs Bit 5 Show Kbus Overrun all other bits are reserved

B.2 PDO Mapping and Assign

B.2.1 Object 0x16nn: RxPDO Mapping of the KBus terminals

For each KBus terminal one RxPDO Mapping object is reserved (see Table 25).

Table 25: KBus Master Objects 0x1600-0x16FE: RxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1600	1	Subindex 001	UINT32	0x7000ppss (pp depends on the output variable, ss is the size of this output variable in bits) If the Output Data size of a variable is greater than 240 bits, only the first 240 bits of the Output Data of the variable are described with the first RxPDO mapping entry of the variable (0x7nn0ppE0), the following PDO mapping entries shall be used to described the mssing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry related to the variable contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	...
	...			
0x1601	1	Subindex 001	UINT32	0x7010ppss
	...			

If a KBus terminal has no Output Data the related RxPDO Mapping object does not exist.

The RxPDO Mapping for each KBus terminal is mandatory for KBus Masters.

B.2.2 Object 0x1Ann: TxPDO Mapping of the KBus Slaves

For each KBus Slave one TxPDO Mapping object is reserved (see Table 26).

Table 26: KBus Master Objects 0x1A00-0x1AFE: TxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1A00	1	Subindex 001	UINT32	0x6000ppss (pp depends on the input variable, ss is the size of this input variable in bits) If the Input Data size of a variable is greater than 240 bits, only the first 240 bits of the Input Data of the variable are described with the first TxPDO mapping entry of the variable (0x6nn0ppE0), the following PDO mapping entries shall be used to described the mssing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry related to the variable contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	...
	...			
0x1A01	1	Subindex 001	UINT32	0x6010ppss
	...			

If a KBus terminal has no Input Data the related TxPDO Mapping object does not exist.

The TxPDO Mapping for each KBus terminal is mandatory for KBus Masters.

B.2.3 Object 0x16FF: RxPDO Mapping of the KBus Control

For the KBus master one RxPDO Mapping object (see Table 27) is reserved, too.

Table 27: KBus Master Object 0x16FF: RxPDO Mapping of KBus Control

Index	Subindex	Name	Data Type	Description / Value
0x16FF	1	Subindex 001	UINT32	0xF2001010

B.2.4 Object 0x1AFF: TxPDO Mapping of the KBus Status

For the KBus master one TxPDO Mapping object (see Table 28) is reserved, too.

Table 28: KBus Master Object 0x1AFF: TxPDO Mapping of KBus Status

Index	Subindex	Name	Data Type	Description / Value
0x1AFF	1	Subindex 001	UINT32	0xF1001010

B.2.5 Object 0x1C2: RxPDO Assign

The KBus Master's RxPDO will be assigned first. After that the RxPDOs of the non-digital terminals shall follow. For alignment the RxPDO 257 shall be assigned after that, if necessary. Then the RxPDOs of the digital terminals shall follow. For alignment the RxPDO 258 shall be assigned after the digital terminals, if necessary.

B.2.6 Object 0x1C13: TxPDO Assign

The KBus Master's TxPDO will be assigned first. After that the TxPDOs of the non-digital terminals shall follow. For alignment the TxPDO 257 shall be assigned after that, if necessary. Then the TxPDOs of the digital terminals shall follow. For alignment the TxPDO 258 shall be assigned after the digital terminals, if necessary.

B.3 EtherCAT State Machine

Table 29 shows the behavior of the KBus Master in the different EtherCAT states.

Table 29: EtherCAT state behavior for the KBus Master fieldbus gateways

EtherCAT State	Behavior / Allowed Operations
INIT	Detect the connected KBus terminals Create the objects 0x6nn0, 0x7nn0
PRE-OP	Optional: download of PDO Mapping Optional: download of PDO Assign Optional: register access to the KBus terminals via the AoE services
SAFE-OP	The output data is in the safe state The input data will be updated with the received Input Data Optional: register access to the KBus terminals via the AoE services
OP	The output data has process values as received via EtherCAT The input data will be updated with the received Input Data Optional: register access to the KBus terminals via the AoE services

B.4 Acyclic Access to the KBus terminals

The KBus Master should support the AoE services to have register access to the KBus terminals. Table 30 shows the relation of the AoE parameter to the KBus.

Table 30: KBus Master Relation of the AoE parameter

AoE Parameter	KBus Relation
Port	1000 + Position of terminal (starting with 0)
Index Group, Bit 8-31 = 0	Register Access
Command	2 = Read Register, 3 = Write Register
Index Group, Bit 0-1	Channel Number
Index Group, Bit 2-7	0
Index Offset, Bit 0-5	Register Number
Index Offset, Bit 6-31	0
Index Group, Bit 8-31 = 1	parameter access
Command	2 = Parameter Read, 3 = Parameter Write
Index Group, Bit 0-1	Channel Number
Index Group, Bit 2-7	0
Index Offset, Bit 0-9	Parameter Number
Index Offset, Bit 10-31	0

B.5 Synchronization

The KBus Master shall support at the least one the synchronization modes as defined in [6].

B.5.1 FreeRun Mode

In the FreeRun Mode the EtherCAT cycle is not synchronized with the KBus cycle.

B.5.2 Synchron Mode

In the Synchron Mode the EtherCAT cycle is synchronized with the KBuscycle. The KBus Cycle Time shall be equal the EtherCAT Cycle time.

B.5.3 DC Mode

In DC Mode the EtherCAT cycle is synchronized with the KBus cycle via Distributed Clocks. The KBus Cycle Time shall be equal the EtherCAT Cycle time.

Annex C PROFIBUS DP Master (Module Profile Number 3100)

The PROFIBUS DP Master gateway is based on the modular device profile (fieldbus gateway). The PROFIBUS DP Master gateway supports up to 125 modules (PROFIBUS DP Slaves).

C.1 Object Dictionary of the PROFIBUS DP Master

The object dictionary of the device contains the object areas as shown in Table 31.

Table 31: PROFIBUS DP Master Object Dictionary Structure

Index	Object Dictionary Area
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x6000...0x6FFF	Input Area (Input Data of the PROFIBUS DP slaves)
0x7000...0x7FFF	Output Area (Output Data of the PROFIBUS DP slaves)
0x8000...0x8FFF	Configuration Area (Expected configuration of the PROFIBUS DP slaves)
0x9000...0x9FFF	Information Area (Detected configuration of the PROFIBUS DP slaves)
0xF000...0xFFFF	Device Area

C.1.1 Object Areas of the PROFIBUS DP Slaves (Modules)

C.1.1.1 Output Data

The Output Data of the PROFIBUS DP master represent the Output Data of the PROFIBUS DP Slaves. Each entry of these objects is related to a DP Module, if a DP Module does not have Output Data, the corresponding entry does not exist.

DP Output Data Slave yyy

Table 32: PROFIBUS DP Master Objects 0x7nn0: DP Output Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x7000	1	DP Module 001	OCTET-STRING	RO	Represents the Output Data of the first DP Module of the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01), if this DP Module does not have Output Data this entry is not existing
	2	DP Module 002	OCTET-STRING	RO	Represents the Output Data of the second DP Module of the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01), if this DP Module does not have Output Data this entry is not existing
..					
0x7000	244	DP Module 244	OCTET-STRING	RO	Represents the Output Data of the 244th DP Module of the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01), if this DP Module does not have Output Data this entry is not existing
..					
0x7010	1	DP Module 001	OCTET-STRING	RO	Represents the Output Data of the first DP Module of the second PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:02), if this DP Module does not have Output Data this entry is not existing
...		...			

If a DP Module does not have Output Data, the related entry does not exist and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex doesn't exist).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

The objects 0x7nn8...0x7nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x7nn1...0x7nn7 are reserved for profile extensions.

C.1.1.2 Input Data

The Input Data of the PROFIBUS DP master represent the Input Data of the PROFIBUS DP Slaves. Each entry of these objects is related to a DP Module, if a DP Module does not have Input Data, the corresponding entry does not exist.

DP Input Data Slave yyy

Table 33: PROFIBUS DP Master Objects 0x6nn0: DP Input Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x6000	1	DP Module 001	OCTET-STRING	RO	Represents the Input Data of the first DP Module of the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01), if this DP Module does not have Input Data this entry is not existing
	2	DP Module 002	OCTET-STRING	RO	Represents the Input Data of the second DP Module of the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01), if this DP Module does not have Input Data this entry is not existing
..					
0x6000	244	DP Module 244	OCTET-STRING	RO	Represents the Input Data of the 244th DP Module of the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01), if this DP Module does not have Input Data this entry is not existing
..					
0x6010	1	DP Module 001	OCTET-STRING	RO	Represents the Input Data of the first DP Module of the second PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:02), if this DP Module does not have Input Data this entry is not existing
...		...			

If a DP Module does not have Input Data, the related entry does not exist and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex doesn't exist).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

The objects 0x6nn8...0x6nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x6nn1...0x6nn7 are reserved for profile extensions.

C.1.1.3 Configuration Data

The Configuration Data contain the PROFIBUS DP communication parameter the PROFIBUS DP Master needs to establish a PROFIBUS DP connection to the PROFIBUS DP Slave.

PROFIBUS DP Communication Parameter Slave yyy

Table 34: PROFIBUS DP Master Objects 0x8nn0: PROFIBUS DP Communication Parameter

Index	Subindex	Name	Data Type	Access	Description / Value
0x8000	1	Station Address	UINT16	RW	Station Address of the first PROFIBUS DP Slave (same value as 0xF020:01)
	4	Ident Number	UINT32	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Expected Ident Number of the first PROFIBUS DP Slave
	29	Network Flags	UINT16	RW	Network Flags to route vendor specific services over PROFIBUS; shall be 0, if not used
	30	Network Port	UINT16	RW	Network Port to route vendor specific services over PROFIBUS; shall be 0, if not used
	31	Network Segment Address	OCTET-STRING[6]	RW	Network Segment Address to route vendor specific services over PROFIBUS; shall be 0, if not used
	32	SI_Flag	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Bit 0: reserved Bit 1: Extra_Alarm_SAP Bit 2: DPV1_Data_Types Bit 3: DPV1_Supported Bit 4: Publisher_Enable Bit 5: Fail_Safe Bit 6: New_Prm Bit 7: Active
	33	Slave_Type	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), 0 = DP-Slave
	34	Max_Diag_Data_Len	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Maximum DP Diag Data length supported by the slave (6...244)
	35	Max_Alarm_Len	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Maximum length of an alarm in the DP Diag Data
	36	Max_Channel_Data_Length	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Maximum length of a DPV1-C1 service PDU (4...244)
	37	Diag_Upd_Delay	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Number of Slave_Diag responses with Prm_Req=1 allowed after sending Set_Prm and Chk_Cfg
	38	Alarm_Mode	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Maximum number of possible alarms
	39	Add_SI_Flag	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Bit 0: NA_To_Abort Bit 1: Ignore_AClr Bit 2-7: reserved
	40	C1_Timeout	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Response Timeout for DPV1-C1 services in 10 ms

Index	Subindex	Name	Data Type	Access	Description / Value
	41-n			RW	32 bytes of vendor specific configuration data; shall be 0, when accessing with CompleteAccess=TRUE, if not used
..					
0x8010	1	Station Address	UINT16	RW	Station Address of the second PROFIBUS DP Slave (same value as 0xF020:02)
...		...			

The DP communication parameter objects 0x8nn0 are mandatory for a PROFIBUS DP Master.

The PROFIBUS DP Master shall automatically update the object 0xF020 when the objects 0x8nn0 are written by the EtherCAT master.

Prm_Data Slave yyy

Table 35: PROFIBUS DP Master Objects 0x8nn1: Prm_Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x8001	--	Prm_Data Slave yyy	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Prm_Data sent to the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01) during DP StartUp with the Set_Prm service
0x8011	--	Prm_Data Slave yyy	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Prm_Data sent to the second PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:02) during DP StartUp with the Set_Prm service
...					

The Prm_Data objects 0x8nn1 are mandatory for a PROFIBUS DP Master.

Cfg_Data Slave yyy

Table 36: PROFIBUS DP Master Objects 0x8nn2: Cfg_Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x8002	0	Cfg_Data Slave yyy	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Cfg_Data sent to the first PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:01) during DP StartUp with the Chk_Cfg service
0x8012	0	Cfg_Data Slave yyy	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Cfg_Data sent to the second PROFIBUS DP Slave (PROFIBUS Station Address is defined in 0xF020:02) during DP StartUp with the Chk_Cfg service
...					

The Cfg_Data objects 0x8nn2 are mandatory for a PROFIBUS DP Master.

PROFIBUS DPV1 Class 2 Communication Parameter Slave yyy
Table 37: PROFIBUS DP Master Objects 0x8nn3: PROFIBUS DPV1 Class 2 Communication Parameter

Index	Subindex	Name	Data Type	Access	Description / Value
0x8003	1	C2_Response_Timeout	UINT16	RW	Response Timeout for DPV1-C2 services in 10 ms
	2	C2_Send_Timeout	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Send Timeout for the connection monitoring
	3	C2_Max_Data_Len	UINT16	RW	Maximum length of a DPV1-C2 service PDU (4-224)
	4	Features_Supported_1	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	5	Features_Supported_2	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	6	Profile_Features_Supported_1	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	7	Profile_Features_Supported_2	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	8	Profile_Ident_Number	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	9	S_Type	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	10	S_Len	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	11	D_Type	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	12	D_Len	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	13	S_API	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	14	S_SCL	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	15	S_Network_Address	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU, only exists if S_Type = 1
	16	S_MAC_Address	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU, only exists if S_Type = 1 and S_Len > 8
	17	D_API	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU
	18	D_SCL	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU

Index	Subindex	Name	Data Type	Access	Description / Value
	19	D_Network_Address	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU, only exists if D_Type = 1
	20	D_MAC_Address	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Service parameter of the DPV1 C2 Initiate PDU, only exists if D_Type = 1 and D_Len > 8
..					
0x8010	1	Station Address	UINT16	RW	Station Address of the second PROFIBUS DP Slave (same value as 0xF020:02)
...		...			

The DPV1 Class 2 communication parameter objects 0x8nn3 are optional for a PROFIBUS DP Master.

The objects 0x8nn8-0x8nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x8nn4-0x8nn7 are reserved for profile extensions.

C.1.1.4 Information Data

The Information Data contain the informations about the connected PROFIBUS DP slaves. Data will be available when the Detect Modules Command is executed or will be updated automatically by the PROFIBUS DP Master. The Information Data are optional for a PROFIBUS DP Master.

Detected PROFIBUS DP Identification Slave yyy

Table 38: PROFIBUS DP Master Objects 0x9nn0: Detected PROFIBUS DP Identification

Index	Subindex	Name	Data Type	Access	Description / Value
0x9000	1	Station Address	UINT16	RO	Station Address of the first PROFIBUS DP Slave found (same value as 0xF040:01)
	4	Ident Number	UINT32	RO	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Ident_Number of the first PROFIBUS DP Slave found
..					
0x9010	1	Station Address	UINT16	RO	Station Address of the second PROFIBUS DP Slave found (same value as 0xF040:02)
...		...			

Real_Cfg_Data Slave yyy

Table 39: PROFIBUS DP Master Objects 0x9nn2: Real_Cfg_Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x9002	0	Real_Cfg_Data Slave yyy	OCTET-STRING	RO	Cfg_Data read with the Get_Cfg service from the first PROFIBUS DP-Slave found (PROFIBUS DP Station Address is defined in 0xF040:01)
	...				
0x9012	0	Real_Cfg_Data Slave yyy	OCTET-STRING[12]	RO	Cfg_Data read with the Get_Cfg service from the second PROFIBUS DP-Slave found (PROFIBUS DP Station Address is defined in 0xF040:02)
	...				

The objects 0x9nn8...0x9nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x9nn1 and 0x9nn3...0x9nn7 are reserved for profile extensions.

C.1.1.5 Diagnosis Data

The Diagnosis Data contain the status and diagnosis information of the PROFIBUS DP slaves or the connection to the PROFIBUS DP slaves. The Diagnosis Data are optional for a PROFIBUS DP Master.

Diag_Data Slave yyy

Table 40: PROFIBUS DP Master Objects 0xAnn2: Diag_Data

Index	Subindex	Name	Data Type	Access	Description / Value
0xA002	--	Diag_Data Slave yyy	OCTET-STRING	RO	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Diag_Data read with the Slave_Diag service from the first PROFIBUS DP-Slave configured (PROFIBUS DP Station Address is defined in 0xF020:01)
0xA012	--	Diag_Data Slave yyy	OCTET-STRING	RO	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Diag_Data read with the Slave_Diag service from the second PROFIBUS DP-Slave configured (PROFIBUS DP Station Address is defined in 0xF020:02)
	...				

The objects 0xAnn0...0xAnn1 and 0xAnn8...0xAnnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0xAnn3...0xAnn7 are reserved for profile extensions.

C.1.2 Object Areas of the Device

The objects of the Device Area shown in Table 41 are standardized for a PROFIBUS DP Master. All other objects in the area 0xF000...0xFFFF are reserved for profile extension.

Table 41: PROFIBUS DP Master Device Object Areas

Index	Object Name
0xF000	Modular device profile
0xF002	Detect Modules Command
0xF010	Module profile list
0xF020	Configured address list
0xF040	Detected address list
0xF101	DP Master Status
0xF102	DP Slave Status
0xF800	DP Bus parameter set

C.1.2.1 Object 0xF000: Modular Device Profile

The Modular device profile object (0xF000) described in Table 42 is mandatory for a PROFIBUS DP Master.

Table 42: PROFIBUS DP Master Object 0xF000: Modular Device Profile

Index	Subindex	Name	Data Type	Access	Description / Value
0xF000	1	Index distance	UINT16	RO	Index distance between two modules = 0x10
	2	Maximum number of modules	UINT16	RO	Maximum number of PROFIBUS DP Slaves connected to the PROFIBUS DP Master = 125
	3	General configuration	UINT32	RO	Available entries in objects 0x8nn0 = 0x00000009
	4	General information	UINT32	RO	Available entries in objects 0x9nn0 = 0x00000009 (if information data supported) or 0x00000000 (if information data not supported)

C.1.2.2 Object 0xF002: Detect Modules Command

The Detect Modules Command object (0xF002) described in Table 43 is optional for a PROFIBUS DP Master.

Table 43: PROFIBUS DP Master Object 0xF002: Detect Modules Command

Index	Subindex	Name	Data Type	Access	Description / Value
0xF002	1	Command Request	OCTET-STRING[2]	RW	Byte 0: Station Address of the PROFIBUS DP Master, Corresponding to IEC 61158, Part 6 (PROFIBUS DP), (-> entry 0xF800:01) Byte 1: Data_Rate, Corresponding to IEC 61158, Part 6 (PROFIBUS DP), (-> entry 0xF800:02) When this subindex is written from the EtherCAT Master, the PROFIBUS DP Master shall be active on the PROFIBUS with the requested Data_Rate and send the Get_Cfg and the Slave_Diag service to the station addresses 0-125 (except the master's station address) to fill the information data
	2	Command Status	UINT8	RO	1: command is finished, no error 3: command is finished, error 100-199: 0-99% of the command is done 255: command is executing
	3	Command Response	OCTET-STRING[n]	RO	Byte 0: like Subindex 2 Byte 1: always 0 Byte 2-3: 0: no error, > 0: vendor specific error code Byte 4: Station Address of the first PROFIBUS DP Slave found Byte 5-6: Ident_Number of the first PROFIBUS DP Slave found Byte 7: Station Address of the second PROFIBUS DP Slave found Byte 8-9: Ident_Number of the second PROFIBUS DP Slave found ...

C.1.2.3 Object 0xF010: Module Profile List

The Module profile list object (0xF010) described in Table 44 is optional for a PROFIBUS DP Master.

Table 44: PROFIBUS DP Master Object 0xF010: Module Profile List

Index	Subindex	Name	Data Type	Access	Description / Value	
0xF010	1	Subindex 001	UINT32	RO	Bit 0-15:	3100
					Bit 16-31	Ident_Number of the first PROFIBUS DP Slave configured (PROFIBUS DP Station Address is defined in 0xF020:01)
...						
	125	Subindex 125	UINT32	RO	Bit 0-15:	3100
					Bit 16-31	Ident_Number of the 125th PROFIBUS DP Slave configured (PROFIBUS DP Station Address is defined in 0xF020:7D)

C.1.2.4 Object 0xF020: Configured Address List

The Configured address list object (0xF020) described in Table 45 is mandatory for a PROFIBUS DP Master.

Table 45: PROFIBUS DP Master Object 0xF020: Configured Address List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF020	1	Subindex 001	UINT16	RO	PROFIBUS DP Station Address of the first PROFIBUS DP Slave configured
	2	Subindex 002	UINT16	RO	PROFIBUS DP Station Address of the second PROFIBUS DP Slave configured
...					
	125	Subindex 125	UINT16	RO	PROFIBUS DP Station Address of the hundred twenty fifth PROFIBUS DP Slave configured

C.1.2.5 Object 0xF040: Detected Address List

The Detected address list object (0xF040) described in Table 46 is optional for a PROFIBUS DP Master.

Table 46: PROFIBUS DP Master Object 0xF040: Detected Address List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF040	1	Subindex 001	UINT16	RO	PROFIBUS DP Station Address of the first PROFIBUS DP Slave found
	2	Subindex 002	UINT16	RO	PROFIBUS DP Station Address of the second PROFIBUS DP Slave found
...					
	126	Subindex 126	UINT16	RO	PROFIBUS DP Station Address of the hundred twenty sixth PROFIBUS DP Slave found

C.1.2.6 Object 0xF101: DP Master Status

The DP Master Status object (0xF101) described in Table 47 is optional for a PROFIBUS DP Master.

Table 47: PROFIBUS DP Master Object 0xF101: DP Master Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF101	1	Bus Error Counter	UINT16	RO	Increments on each bus error
	2	Cycle Counter	UINT16	RO	Increments on each DP Cycle
	3	Slave Status Counter	UINT16	RO	Number of DP slaves that were not exchanging data in the preceding cycle
	4	Cycle Time	UINT16	RO	Cycle Time of the preceding DP cycle in xx
	5	Repeat Counter	UINT16	RO	Number of repeats in the preceding DP Cycle
	20	Device Diag	BOOLEAN	RO	True, if diagnostic information is present in the master
	21	Sync Error	BOOLEAN	RO	True, if a synchronization error occurred in the preceding cycle in DC Mode
	22	Cycle Toggle	BOOLEAN	RO	Shall toggle with each DP Cycle
	23	Cycle State	BOOLEAN	RO	True, if at least one DP Slave is not exchanging data

C.1.2.7 Object 0xF102: DP Slave Status

The DP Slave Status object (0xF102) described in Table 48 is optional for a PROFIBUS DP Master.

Table 48: PROFIBUS DP Master Object 0xF102: DP Slave Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF102	1	DP Status of Slave 1	UINT8	RO	status of communication of first DP Slave
					0 Slave is exchanging data, DP inputs are up to date
					1 Slave is deactivated
					2 Slave does not respond
					3 slave currently exchanging data with another master
					4 incorrect slave response e.g. service not activated in the case of data exchange
					5 Slave reports parametrization error error in the SetPrm telegram (usually Ident-Number or UserPrmData)
					6 Slave reports that a DP function is not supported
					7 Slave reports configuration error 8 error in the ChkCfg telegram (usually incorrect DP module configured)
					8 Slave is in DP bootup but is not yet ready for data exchange
					9 Slave reports static diagnosis
					10 reserved
					11 Resoponse with bus error e.g. parity or checksum error
					12...13 reserved
					14 Resoponse with telegram error e.g. request bit set
					15 Slave reports no resources e.g. PrmData or CfgData too long
					16 Slave reports that the DP service is not activated
					17 Unexpected telegram received e.g. token when waiting for a response from a slave
					18 Slave is ready for data exchange, but no process data are exchanged via EtherCAT yet
					19...255 reserved
	2	DP Status of Slave 2	UINT8	RO	status of communication of second DP Slave
	...				
	125	DP Status of Slave 125	UINT8	RO	status of communication of 125 th DP Slave

C.1.2.8 Object 0xF800: DP Bus Parameter Set

The DP Bus parameter set object (0xF800) described in Table 49 is mandatory for a PROFIBUS DP Master.

Table 49: PROFIBUS DP Master Object 0xF800: DP Bus Parameter Set

Index	Subindex	Name	Data Type	Access	Description / Value
0xF800	1	DL_Add	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), PROFIBUS DP Station Address of the PROFIBUS DP Master (0-125)
	2	Data_rate	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), 0 = 9.6 kBaud 1 = 19.2 kBaud 2 = 93.75 kBaud 3 = 187.5 kBaud 4 = 500 kBaud 6 = 1.5 MBaud 7 = 3 MBaud 8 = 6 MBaud 9 = 12 MBaud 10 = 31.25 kBaud 11 = 45.45 kBaud
	3	Tsl	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Slot Time
	4	Min TSDR	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Min. TSDR
	5	Max TSDR	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Max. TSDR
	6	Tqui	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Quiet Time
	7	Tset	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Setup Time
	8	Ttr	UINT32	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Target Token Rotation Time
	9	G	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), GAP Update Factor
	10	HSA	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Highest Active Station Address
	11	Max_retry_limit	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Max Retry Limit
	12	Bp_Flag	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Bit 0-2: reserved Bit 3-4: Isochronous Mode Bit 5: IsoM Sync Bit 6: IsoM Freeze Bit 7: Error_Action_Flag
	13	Min_Slave_Interval	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Min. Slave Interval Time in 100 µs
	14	Poll_Timeout	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Poll Timeout Time in 1 ms
	15	Data_Control_Time	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Data Control Time in 10 ms
	16	Alarm_Max	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP),
	17	Max_User_Global_Control	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP),

Index	Subindex	Name	Data Type	Access	Description / Value
	18-n			RW	32 bytes of vendor specific configuration data, shall be zero, when accessing with CompleteAccess=TRUE, if not used

C.2 PDO Mapping and Assign

C.2.1 Objects 0x16nn: RxPDO Mapping of the PROFIBUS DP Slaves

For each PROFIBUS DP Slave one RxPDO Mapping object is reserved (see Table 50).

Table 50: PROFIBUS DP Master Objects 0x1600...0x167C: RxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1600	1	Subindex 001	UINT32	0x7000ppss pp depends on the related DP-Module of the first configured PROFIBUS DP Slave (PROFIBUS DP Station Address in entry 0xF020:01), ss is the size of the Output Data of this DP Module in bits, starting with the first DP Module with Output Data of this Slave) If the Output Data size of a DP Module is greater than 240 bits, only the first 240 bits of the Output Data of the DP Module are described with the first RxPDO mapping entry of the DP Module (0x7nn0ppE0), the following PDO mapping shall be used to described the missing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry related to the DP Module contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	...
	...			
0x1601	1	Subindex 001	UINT32	0x7010ppss (pp depends on the related DP-Module of the second configured PROFIBUS DP Slave (PROFIBUS DP Station Address in entry 0xF020:02), ss is the size of the Output Data of this DP Module in bits, starting with the first DP Module with Output Data of this Slave)
	...			

If a PROFIBUS DP Slave has no Output Data the related RxPDO Mapping object does not exist.

The RxPDO Mapping for each PROFIBUS DP Slave is mandatory for PROFIBUS DP Masters.

C.2.2 Objects 0x1Ann: TxPDO Mapping of the PROFIBUS DP Slaves

For each PROFIBUS DP Slave one TxPDO Mapping object is reserved (see Table 51).

Table 51: PROFIBUS DP Master Objects 0x1A00...0x1A7C: TxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1A00	1	Subindex 001	UINT32	0x6000ppss pp depends on the related DP-Module of the first configured PROFIBUS DP Slave (PROFIBUS DP Station Address in entry 0xF020:01), ss is the size of the Input Data of this DP Module in bits, starting with the first DP Module with Input Data of this Slave)
				If the Input Data size of a DP Module is greater than 240 bits, only the first 240 bits of the Input Data of the DP Module are described with the first TxPDO mapping entry of the DP Module (0x6nn0ppE0), the following PDO mapping shall be used to described the missing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry related to the DP Module contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	
	...			
0x1A01	1	Subindex 001	UINT32	0x6010ppss (pp depends on the related DP-Module of the second configured PROFIBUS DP Slave (PROFIBUS DP Station Address in entry 0xF020:02), ss is the size of the Input Data of this DP Module in bits, starting with the first DP Module with Input Data of this Slave)
	...			

If a PROFIBUS DP Slave has no Input Data the related TxPDO Mapping object does not exist.

The TxPDO Mapping for each PROFIBUS DP Slave is mandatory for PROFIBUS DP Masters.

C.2.3 Object 0x1A81: TxPDO Mapping of PROFIBUS PDO State

The PDO State as described in Table 52 is used to signal if the communication between the PROFIBUS DP Master and the PROFIBUS DP Slaves is established. If a PROFIBUS DP Slave has a related TxPDO the TxPDO State is defined, if a PROFIBUS DP Slave only has a related RxPDO the RxPDO State is defined.

Table 52: PROFIBUS DP Master Objects 0x14nn/0x18nn: PROFIBUS PDO State

Description	Index	Subindex	Data Type	Value
PDO State of 1st PROFIBUS DP Slave configured	TxPDO 1 exists: 0x1800 TxPDO 1 do not exist: 0x1400	7	BOOLEAN	0: PROFIBUS DP Slave is in Data_Exchange 1: PROFIBUS DP Slave is not in Data_Exchange
PDO State of 2nd PROFIBUS DP Slave configured	TxPDO 2 exists: 0x1801 TxPDO 2 do not exist: 0x1401	7	BOOLEAN	0: PROFIBUS DP Slave is in Data_Exchange 1: PROFIBUS DP Slave is not in Data_Exchange
...				

The PROFIBUS PDO States of the configured PROFIBUS DP Slaves are mapped to TxPDO Mapping object 0x1A81 as shown in Table 53.

Table 53: PROFIBUS DP Master Object 0x1A81: TxPDO Mapping of PROFIBUS PDO States

Subindex	Name	Data Type	Value
1	Subindex 001	UINT32	0x18000701 (if 0x1A00 exists) Index 0x1800:SI07 = TxPDO State
			0x14000701 (if 0x1A00 does not exist) Index 0x1400:SI07 = RxPDO State
2	Subindex 002	UINT32	0x18010701 (if 0x1A01 exists) 0x14010701 (if 0x1A01 does not exist)
...			

The PDO State is mandatory for PROFIBUS DP Masters.

C.2.4 Object 0x1A82: TxPDO Mapping of PROFIBUS PDO Toggle

The PDO Toggle as described in Table 54 shall toggle with every new receive of DP input data and update of EtherCAT input data.

Table 54: PROFIBUS DP Master Objects 0x14nn/0x18nn: PROFIBUS PDO Toggle

Description	Index	Subindex	Data Type	Value
PDO Toggle of 1st PROFIBUS DP Slave configured	TxPDO 1 exists: 0x1800 TxPDO 1 do not exist: 0x1400	9	BOOLEAN	Toggles with every DP input update cycle of first DP slave
PDO Toggle of 2nd PROFIBUS DP Slave configured	TxPDO 2 exists: 0x1801 TxPDO 2 do not exist: 0x1401	9	BOOLEAN	Toggles with every DP input update cycle of second DP slave
...				

The PROFIBUS PDO Toggle information of the configured PROFIBUS DP Slaves are mapped to TxPDO Mapping object 0x1A82 as shown in Table 55.

Table 55: PROFIBUS DP Master Object 0x1A82: TxPDO Mapping of PROFIBUS PDO Toggle

Subindex	Name	Data Type	Value
1	Subindex 001	UINT32	0x18000701 (if 0x1A00 exists) Index 0x1800:SI07 = TxPDO State 0x14000701 (if 0x1A00 does not exist) Index 0x1400:SI07 = RxPDO State
2	Subindex 002	UINT32	0x18010701 (if 0x1A01 exists) 0x14010701 (if 0x1A01 does not exist)
...			

The TxPDO Mapping of Profibus PDO Toggle is optional for PROFIBUS DP Masters.

C.2.5 Objects 0x1A84: TxPDO Mapping of the PROFIBUS DP Slave Status

The PROFIBUS DP Slave Status (Objet 0xF102) informations are mapped to TxPDO Mapping PROFIBUS DP Slave Status as shown in Table 56.

Table 56: PROFIBUS DP Master Objects 0x1A84: TxPDO Mapping of PROFIBUS DP Slave Status

Subindex	Name	Data Type	Description / Value
1	Subindex 001	UINT32	0xF102:01, 8
2	Subindex 002	UINT32	0xF102:02, 8
...			

The TxPDO Mapping of PROFIBUS DP Slave Status is optional for PROFIBUS DP Masters.

C.2.6 Object 0x1C12: RxPDO Assign

The RxPDOs of the PROFIBUS DP Slaves will be assigned in their order in the RxPDO Assign Object 0x1C12 (see Table 57) followed by optional vendor specific RxPDOs.

Table 57: PROFIBUS DP Master Object 0x1C12: RxPDO Assign

Subindex	Assigned RxPDO
1	RxPDO Mapping Index related to the 1st PROFIBUS DP Slave with Output Data 0x1600
2	RxPDO Mapping Index related to the 2nd PROFIBUS DP Slave with Output Data 0x1601
...	
n	RxPDO Mapping Index related to the last PROFIBUS DP Slave with Output Data
n+1	Vendor specific RxPDO if supported
n+2	...
...	

If a PROFIBUS DP Slave does not have a RxPDO there is no gap in the RxPDO Assign Object.

C.2.7 Object 0x1C13: TxPDO Assign

The TxPDOs of the PROFIBUS DP Slaves will be assigned in their order in the TxPDO Assign Object 0x1C13 (see Table 58) followed by the TxPDO for PDO State and optionally the TxPDO for the TxPDO Toggle and vendor specific TxPDOs.

Table 58: PROFIBUS DP Master Object 0x1C13: TxPDO Assign

Subindex	Assigned TxPDO
1	TxPDO Mapping Index related to the 1st PROFIBUS DP Slave with Input Data 0x1A00
2	TxPDO Mapping Index related to the 2nd PROFIBUS DP Slave with Input Data 0x1A01
...	
n	TxPDO Mapping Index related to the last PROFIBUS DP Slave with Input Data
n+1	0x1A81 (TxPDO with PDO States)
n+2	0x1A82 (if TxPDO Toggle is supported) or vendor specific TxPDO (if TxPDO Toggle is not supported)
n+3	0x1A84 (if TxPDO DP Slave Status supported) or vendor specific TxPDO
...	

If a PROFIBUS DP Slave does not have a TxPDO there is no gap in the TxPDO Assign Object.

C.3 EtherCAT State Machine

Table 59 shows the behavior of the PROFIBUS DP Master in the different EtherCAT states.

Table 59: EtherCAT state behavior for the PROFIBUS DP Master fieldbus gateways

EtherCAT State	Behavior / Allowed Operations
INIT	Not connected to the PROFIBUS (DP Master in OFFLINE mode) If the backup parameter were stored, loading the stored configuration from non-volatile memory
PRE-OP	Connect to the PROFIBUS (DP Master in STOP mode), when the DP Bus Parameter Set object (0xF800) was downloaded completely, a stored configuration was loaded from non-volatile memory or the Detect Modules command was executed Create the objects 0x6nn0, 0x7nn0, 0x8nn0 when the objects 0x8nn0-0x8nn2 were downloaded completely or a stored configuration was loaded from non-volatile memory Optional: download of PDO Mapping Optional: download of PDO Assign Optional: detect the connected fieldbus slaves and fill the information data objects 0x9nn0 if Detect Modules command is executed Optional: DPV1 access to the configured or detected PROFIBUS DP Slaves via the AoE services
SAFE-OP	PROFIBUS DP Slave handler for each configured PROFIBUS DP Slave is started The output data is in the safe state (DP Master in CLEAR Mode) The input data will be updated with the received Input Data, PDO State and TxPDO Toggle Optional: DPV1 access to the configured or detected PROFIBUS DP Slaves via the AoE services
OP	The output data has process values as received via EtherCAT (DP Master in OPERATE Mode) The input data will be updated with the received Input Data, PDO State and TxPDO Toggle Optional: DPV1 access to the configured or detected PROFIBUS DP Slaves via the AoE services

C.4 Device configuration

The Configuration Data objects (0x8000...0x8FFF and 0xF800) can be created in a PROFIBUS DP Master device in three different ways:

1. The objects should be created when they are downloaded completely the first time (with CompleteAccess=TRUE), starting with object 0xF800, followed by 0x8000, 0x8001-0x8002, 0x8010, etc.
2. The objects should be created automatically by detecting the PROFIBUS DP slaves on the PROFIBUS DP bus
3. The objects should be created when they were stored as backup parameter

The PROFIBUS DP Master shall support at least one of the first two methods.

When supporting the first method the PROFIBUS DP Master should support backup parameters because

1. the number of objects to be downloaded in PRE-OP delays the EtherCAT Start-Up that it will only be done if something was changed

2. the setting of the complete PROFIBUS DP configuration is very complex and could be done from a vendor specific tool via the mailbox gateway

C.5 Acyclic Access to the DP Slaves

The PROFIBUS DP Master should support the AoE services to have DPV1 access to the configured PROFIBUS DP slaves. The Relation of AoE parameter to PROFIBUS DP is shown in Table 60.

Table 60: PROFIBUS DP Master Relation of the AoE parameter

AoE Parameter		PROFIBUS DP Relation
Port		1000 + Station Address of the PROFIBUS DP Slave
Index Group, Bit 8-31 = 0		DPV1 C1 (only if 0x8nn0:20h, Bit 3 is set and 0x8nn1[9], Bit 7 is set)
	Command	2 = DPV1 Read, 3 = DPV1 Write, 9 = DPV1 Data_Transport
	Index Group, Bit 0-7	Slot Number
	Index Offset, Bit 0-7	Index
	Index Offset, Bit 8-15	Response Timeout (optional) for actual request in 100 ms (0 means configured response timeout (-> 0x8nn0:28h))
	Index Offset, Bit 16-31	Reserved
Index Group, Bit 8-31 = 1		DPV1 C2 (only if object 0x8nn3 was downloaded)
	Command	2 = DPV1 Read, 3 = DPV1 Write, 9 = DPV1 Data_Transport
	Index Group, Bit 0-7	Slot Number
	Index Offset, Bit 0-7	Index
	Index Offset, Bit 8-15	Response Timeout (optional) for actual request in 100 ms (0 means configured response timeout (-> 0x8nn3:01h))
	Index Offset, Bit 16-31	Reserved
Index Group, Bit 8-31 = 2-0xFFFFF		Vendor_Specific

C.6 Synchronization

The PROFIBUS DP Master shall support at the least one the synchronization modes as defined in [6].

C.6.1 FreeRun Mode

In the FreeRun Mode the EtherCAT cycle is not synchronized with the PROFIBUS DP Master's cycle.

C.6.2 Synchron Mode

In the Synchron Mode the EtherCAT cycle is synchronized with the PROFIBUS DP Master's cycle. The DP Cycle Time shall be equal the EtherCAT Cycle time.

C.6.3 DC Mode

In DC Mode the EtherCAT cycle is synchronized with the PROFIBUS DP Master's cycle via Distributed Clocks. If the Isochron Mode is activated, the Isochronous Global_Control service shall be sent at the beginning of the PROFIBUS DP Master's cycle. The DP Cycle Time shall be equal the EtherCAT Cycle time.

Annex D PROFIBUS DP Slave (Module Profile Number 3110)

The PROFIBUS DP Slave gateway is based on the modular device profile (fieldbus gateway). The PROFIBUS DP Slave gateway supports 1 module. The PROFIBUS DP Slave module could be integrated in the PROFIBUS Master (see chapter Annex C). In that case the PROFIBUS DP Slave shall be the first module of the PROFIBUS DP Master.

D.1 Object Dictionary of the PROFIBUS DP Slave Device

The object dictionary of the device contains the object areas as described in Table 61.

Table 61: PROFIBUS DP Slave device Object Dictionary Structure

Index	Object Dictionary Area
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x6000...0x6FFF	Input Area (Input Data of the PROFIBUS DP slave)
0x7000...0x7FFF	Output Area (Output Data of the PROFIBUS DP slave)
0x8000...0x8FFF	Configuration Area (Configuration of the PROFIBUS DP slave)
0xF000...0xFFFF	Device Area

D.1.1 Object Areas of the PROFIBUS DP Slave (Module)

D.1.1.1 Output Data

The Output Data of the PROFIBUS DP Slave module represent the DP Input Data of the PROFIBUS DP Slave. Each entry of these objects is related to a DP Module, if a DP Module does not have Input Data, the corresponding entry does not exist. The Output Data is mandatory for a PROFIBUS DP Slave.

DP Input Data

Table 62: PROFIBUS DP Slave Object 0x7000: DP Input Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x7000	1	DP Module 001	OCTET-STRING	RO	Represents the DP Input Data of the first DP Module of the PROFIBUS DP Slave, if this DP Module does not have DP Input Data this entry is not existing
	2	DP Module 002	OCTET-STRING	RO	Represents the DP Input Data of the second DP Module of the PROFIBUS DP Slave, if this DP Module does not have DP Input Data this entry is not existing
..					
0x7000	244	DP Module 244	OCTET-STRING	RO	Represents the DP Input Data of the 244th DP Module of the PROFIBUS DP Slave, if this DP Module does not have DP Input Data this entry is not existing

If a DP Module does not have DP Input Data, the related entry does not exist and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex doesn't exist).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

When the PROFIBUS DP Slave does not have DP Input Data, this object does not exist and a SDO Upload with CompleteAccess = TRUE or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object doesn't exist).

The objects 0x7nn8-0x7nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x7nn1-0x7nn7 are reserved for profile extensions.

D.1.1.2 Input Data

The Input Data of the PROFIBUS DP Slave module represent the DP Output Data of the PROFIBUS DP Slave. Each entry of these objects is related to a DP Module, if a DP Module does not have DP

Output Data, the corresponding entry does not exist. The Input Data is mandatory for a PROFIBUS DP Slave.

DP Output Data

Table 63: PROFIBUS DP Slave Objects 0x6000: DP Output Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x6000	1	DP Module 001	OCTET-STRING	RO	Represents the DP Output Data of the first DP Module of the PROFIBUS DP Slave, if this DP Module does not have DP Output Data this entry is not existing
	2	DP Module 002	OCTET-STRING	RO	Represents the DP Output Data of the second DP Module of the PROFIBUS DP Slave, if this DP Module does not have DP Output Data this entry is not existing
..					
0x6000	244	DP Module 244	OCTET-STRING	RO	Represents the DP Output Data of the 244th DP Module of the PROFIBUS DP Slave, if this DP Module does not have DP Output Data this entry is not existing

If a DP Module does not have DP Output Data, the related entry does not exist and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

When the PROFIBUS DP Slave does not have DP Output Data, the related object does not exist and a SDO Upload with CompleteAccess = TRUE or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The objects 0x6nn8-0x6nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x6nn1-0x6nn7 are reserved for profile extensions.

D.1.1.3 Configuration Data

The Configuration Data contain the PROFIBUS DP communication parameter.

PROFIBUS DP communication parameter

The DP communication parameter object 0x8000 described in Table 64 is mandatory for a PROFIBUS DP Slave.

Table 64: PROFIBUS DP Slave Objects 0x8000: PROFIBUS DP Communication Parameter

Index	Subindex	Name	Data Type	Access	Description / Value
0x8000	1	Station Address	UINT16	RW	Station Address of the PROFIBUS DP Slave (same value as 0xF800:01)
	4	Ident Number	UINT32	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Ident Number of the PROFIBUS DP Slave
	29	Network Flags	UINT16	RW	Network Flags to route vendor specific services over PROFIBUS; Shall be 0, if not used
	30	Network Port	UINT16	RW	Network Port to route vendor specific services over PROFIBUS; Shall be 0, if not used
	31	Network Segment Address	OCTET-STRING[6]	RW	Network Segment Address to route vendor specific services over PROFIBUS; Shall be 0, if not used
	32		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	33		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	34		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	35		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	36		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	37		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	38		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	39		UINT8	RW	Reserved for PROFIBUS DP Master; shall be 0
	40		UINT16	RW	Reserved for PROFIBUS DP Master; shall be 0
	41-n			RW	32 bytes of vendor specific configuration data; Shall be zero if not supported by Master (Write) or Slave (Read) and access with CompleteAccess=TRUE

Real_Cfg_Data

The Cfg_Data object 0x8002 shown in Table 65 is mandatory for a PROFIBUS DP Slave.

Table 65: PROFIBUS DP Slave Object 0x8002. Real_Cfg_Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x8002	0	Real_Cfg_Data	OCTET-STRING	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Real_Cfg_Data of the PROFIBUS DP Slave

If this object is downloaded during PRE-OP from the EtherCAT Master, the Cfg_Data received from the PROFIBUS DP Master should match exactly. If this object is not downloaded during PRE-OP from the EtherCAT Master, only the DP Input and Output Sizes should match with the EtherCAT Output and Input Sizes and the Real_Cfg_Data should be updated with the Cfg_Data sent from the PROFIBUS DP Master.

D.1.2 Object Areas of the Device

The objects of the Device Area shown in Table 66 are standardized for a PROFIBUS DP Slave. These objects shall be compatible to a PROFIBUS DP Master because the PROFIBUS DP Slave could be integrated in the PROFIBUS DP Master. All other objects in the area 0xF000-0xFFFF are reserved for profile extensions.

Table 66: PROFIBUS DP Slave Device Object Areas

Index	Object Name
0xF000	Modular device profile
0xF100	DP Status
0xF800	DP Bus parameter set

D.1.2.1 Object 0xF000: Modular Device Profile

The Modular device profile object (0xF000) shown in Table 67 is mandatory for a PROFIBUS DP Slave.

Table 67: PROFIBUS DP Slave Object 0xF000: Modular Device Profile

Index	Subindex	Name	Data Type	Access	Description / Value
0xF000	1	Index distance	UINT16	RO	Index distance between two modules = 0x10
	2	Maximum number of modules	UINT16	RO	Number of Modules = 1
	3	General configuration	UINT32	RO	Available entries in objects 0x8nn0 = 0x00000009

D.1.2.2 Object 0xF100: DP Status

The DP status object (0xF100) shown in Table 68 is optional for a PROFIBUS DP Slave.

Table 68: PROFIBUS DP Slave Object 0xF100: DP Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF100	1	DP Communication Status	ENUM[8]	RO	Status of the DP connection to the DP Master
					127 Baudrate search
					128 WAIT-PRM
					129 WAIT-CFG
					130 DATA-EXCH, CLEAR
					0 DATA-EXCH - OPERATE
	9		BIT6		Reserved for future use
	15	TxPDO Toggle	BOOL	RO	same value as 0x1800:09; will toggle every time the DP Outputs have been updated by the PROFIBUS DP Master
	16	TxPDO State	BOOL	RO	same value as 0x1800:07; will be set, if the Input Data is not valid (0xF100:01 > 0)

D.1.2.3 Object 0xF800: DP Bus Parameter Set

The DP Bus Parameter Set object (0xF800) described in Table 69 is optional for a PROFIBUS DP Slave. If this object is not supported or not downloaded by the EtherCAT master during PRE-OP, the PROFIBUS DP Slave shall use the default values of this object.

Table 69: PROFIBUS DP Slave Object 0xF800: DP Bus Parameter Set

Index	Subindex	Name	Data Type	Access	Description / Value
0xF800	1	DL_Add	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), PROFIBUS DP Station Address of the PROFIBUS DP Slave (0-125); same value as 0x8000:01
	2	Data_rate	UINT8	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), 0 = 9.6 kBaud 1 = 19.2 kBaud 2 = 93.75 kBaud 3 = 187.5 kBaud 4 = 500 kBaud 6 = 1.5 MBaud 7 = 3 MBaud 8 = 6 MBaud 9 = 12 MBaud 10 = 31.25 kBaud 11 = 45.45 kBaud 255 = Auto-Baud (default)
	3		UINT16	RW	Reserved for PROFIBUS DP Master
	4	Min TSDR	UINT16	RW	Corresponding to IEC 61158, Part 6 (PROFIBUS DP), Min. TSDR (default value: 11)
	5		UINT16	RW	reserved for PROFIBUS DP Master
	6		UINT8	RW	reserved for PROFIBUS DP Master
	7		UINT8	RW	reserved for PROFIBUS DP Master
	8		UINT32	RW	reserved for PROFIBUS DP Master
	9		UINT8	RW	reserved for PROFIBUS DP Master
	10		UINT8	RW	reserved for PROFIBUS DP Master
	11		UINT8	RW	reserved for PROFIBUS DP Master
	12		UINT8	RW	reserved for PROFIBUS DP Master
	13		UINT16	RW	reserved for PROFIBUS DP Master
	14		UINT16	RW	reserved for PROFIBUS DP Master
	15		UINT16	RW	reserved for PROFIBUS DP Master
	16		UINT8	RW	reserved for PROFIBUS DP Master
	17		UINT8	RW	reserved for PROFIBUS DP Master
	18-n			RW	32 bytes of vendor specific configuration data; Shall be zero if not supported by Master (Write) or Slave (Read) and access with CompleteAccess=TRUE

D.2 PDO Mapping and Assign

D.2.1 Object 0x1600: RxPDO Mapping of the PROFIBUS DP Slave

For the PROFIBUS DP Slave one RxPDO Mapping object is reserved (see Table 70).

Table 70: PROFIBUS DP Slave Object 0x1600: RxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1600	1	Subindex 001	UINT32	0x7000ppss (pp depends on the related DP-Module of the PROFIBUS DP Slave, ss is the size of the DP Input Data of this DP Module in bits, starting with the first DP Module with DP Input Data of the PROFIBUS DP Slave) If the DP Input Data size of a DP Module is greater than 240 bits, only the first 240 bits of the DP Input Data of the DP Module are described with the first RxPDO mapping entry of the DP Module (0x7nn0ppE0), the following PDO mapping shall be used to described the missing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry related to the DP Module contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	...
	...			

If the PROFIBUS DP Slave has no Output Data this RxPDO Mapping object does not exist.

The RxPDO Mapping for the PROFIBUS DP Slave is mandatory for the PROFIBUS DP Slave Gateway.

D.2.2 Object 0x1A00: TxPDO Mapping of the PROFIBUS DP Slave

For the PROFIBUS DP Slave one TxPDO Mapping object is reserved (see Table 71).

Table 71: PROFIBUS DP Slave Object 0x1A00: TxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1A00	1	Subindex 001	UINT32	0x6000ppss (pp depends on the related DP-Module of the PROFIBUS DP Slave, ss is the size of the DP Output Data of this DP Module in bits, starting with the first DP Module with DP Output Data of this PROFIBUS DP Slave) If the DP Output Data size of a DP Module is greater than 240 bits, only the first 240 bits of the DP Output Data of the DP Module are described with the first TxPDO mapping entry of the DP Module (0x6nn0ppE0), the following PDO mapping shall be used to described the missing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry related to the DP Module contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	...
	...			

If the PROFIBUS DP Slave has no Input Data this TxPDO Mapping object does not exist.

The TxPDO Mapping for the PROFIBUS DP Slave is mandatory for PROFIBUS DP Slave Gateways.

D.2.3 Object 0x1A7F: TxPDO Mapping of PROFIBUS DP Status

The Status PDO is mandatory for the PROFIBUS DP Slave, if the mapped entries are not supported, the corresponding PDO mapping entries shall contain the align information.

Table 72: PROFIBUS DP Slave Object 0x1A7F: TxPDO Mapping of PROFIBUS DP Status

Subindex	Name	Data Type	Value
1	Subindex 001	UINT32	0xF1000108
2	Subindex 002	UINT32	0x00000006
2	Subindex 003	UINT32	0x18000901
2	Subindex 004	UINT32	0x18000701

D.2.4 Object 0x1C12: RxPDO Assign

The RxPDO of the PROFIBUS DP Slave will be assigned first in the RxPDO Assign Object 0x1C12 (see Table 73) followed by optional vendor specific RxPDOs.

Table 73: PROFIBUS DP Slave Object 0x1C12RxPDO Assign

Subindex	Assigned RxPDO
1	0x1600
2	Vendor specific RxPDO if supported
3	...
...	

If the PROFIBUS DP Slave does not have a RxPDO there is no gap in the RxPDO Assign Object.

D.2.5 Object 0x1C13: TxPDO Assign

The Status PDO will be assigned first, following the TxPDO of the PROFIBUS DP Slave and vendor specific TxPDOs as described in Table 74.

Table 74: PROFIBUS DP Slave Object 0x1C13TxPDO Assign

Subindex	Assigned TxPDO
1	0x1A00
2	0x1A7F
3	Vendor specific TxPDO if supported
...	

If the PROFIBUS DP Slave does not have a TxPDO there is no gap in the TxPDO Assign Object.

D.3 EtherCAT State Machine

Table 75 shows the behavior of the PROFIBUS DP Slave in the different EtherCAT states.

Table 75: EtherCAT state behavior for the PROFIBUS DP Slave fieldbus gateways

EtherCAT State	Behavior / Allowed Operations
INIT	Not connected to the PROFIBUS If the backup parameter were stored, loading the stored configuration from non-volatile memory
PRE-OP	Connect to the PROFIBUS (but DP is still deactivated), when the DP Bus Parameter Set object (0xF800) or the DP communication parameter object (0x8000) was downloaded completely or a stored configuration was loaded from non-volatile memory Create the objects 0x6000, 0x7000, 0x8000 and 0x8002 when the objects 0x8000 and 0x8002 were downloaded completely or a stored configuration was loaded from non-volatile memory Optional: download of PDO Mapping Optional: download of PDO Assign
SAFE-OP	Activate the DP communication If neither the object 0x8002 was not downloaded, the objects 0x6000, 0x7000 and 0x8002 will be created depending on the received Cfg_Data of the DP master The Static Diagnosis Bit is set, if the PROFIBUS DP Slave has DP Input Data because no valid DP Input Data (EtherCAT Output Data are in safe state) can be sent The TxPDO State is set, if the PROFIBUS DP Slave has DP Input Data
OP	The DP Input Data have process values as received via EtherCAT Output Data The EtherCAT Input Data will be updated with the received DP Output Data The TxPDO State and TxPDO Toggle will be updated The Static Diagnosis Bit will be reset

D.4 Device configuration

The Configuration Data objects (0x6000, 0x7000, 0x8000 and 0x8002) can be created in a PROFIBUS DP Slave device in three different ways:

1. The objects should be created when they are downloaded completely the first time (with CompleteAccess=TRUE), starting with object 0xF800 (optional), followed by 0x8000 and 0x8002
2. The objects should be created automatically when switching to SAFE-OP if they don't exist, in that case the objects will be updated according to the received Cfg_Data (if the input and output sizes match)
3. The objects should be created when they were stored as backup parameter

The PROFIBUS DP Slave has to support at least one of the first two methods.

D.5 Synchronization

The PROFIBUS DP Slave shall support the Free Run Mode as defined in [6]

D.5.1 FreeRun Mode

In the FreeRun Mode the EtherCAT cycle is not synchronized with the PROFIBUS DP cycle.

Annex E Interbus Slave (Module Profile Number 4010)

E.1 General

The Interbus Slave gateway is based on the modular device profile (fieldbus gateway). The Interbus Slave gateway supports 1 module.

E.2 Object Dictionary of the Interbus Slave Device

The object dictionary of the device contains the object areas described in Table 76.

Table 76: Interbus Slave device Object Dictionary Structure

Index	Object Dictionary Area
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x6000...0x6FFF	Input Area (Input Data of the Interbus slave)
0x7000...0x7FFF	Output Area (Output Data of the Interbus slave)
0x8000...0x8FFF	Configuration Area (Configuration of the Interbus slave)
0xF000...0xFFFF	Device Area

E.2.1 Object Areas of the Interbus Slave (Module)

E.2.1.1 Output Data

The Output Data of the Interbus Slave module represent the Interbus Data sent to the Interbus Master. The Output Data is mandatory for an Interbus Slave.

Interbus Send Data

Table 77 contains the Object 0x7000 of the Interbus Slave.

Table 77: Interbus Slave Object 0x7000: Interbus Send Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x7000	1	Subindex 001	OCTET-STRING	RO	Represents the Interbus Data sent to the Interbus Master

When the Interbus Slave does not have Interbus Data sent to the Interbus Master, this object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The objects 0x7nn8...0x7nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x7nn1...0x7nn7 are reserved for profile extensions.

E.2.1.2 Input Data

The Input Data of the Interbus Slave module represent the Interbus Data received from the Interbus Master. The Input Data is mandatory for an Interbus Slave.

Interbus Receive Data

Table 78 contains the Object 0x6000 of the Interbus Slave.

Table 78: Interbus Slave Object 0x6000: Interbus Receive Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x6000	1	Subindex 001	OCTET-STRING	RO	Represents the DP Interbus Data received from the Interbus Master

When the Interbus Slave does not have Interbus Data received from the Interbus Master, the related object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The objects 0x6nn1, 0x6nn8-0x6nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x6nn2...0x6nn7 are reserved for profile extensions.

E.2.2 Object Areas of the Device

The objects of the Device Area described in Table 79 are standardized for an Interbus Slave. All other objects in the area 0xF000-0xFFFF are reserved for profile extensions.

Table 79: Interbus Slave Object Areas of the Device

Index	Object Name
0xF000	Modular device profile
0xF100	Interbus Status
0xF800	Interbus Parameter Setting
0xF801	Interbus Diagnosis

E.2.2.1 Object 0xF000: Modular Device Profile

The Modular device object (0xF000) shown in Table 80 is mandatory for an Interbus Slave.

Table 80: Interbus Slave Object 0xF000: Modular Device Profile

Index	Subindex	Name	Data Type	Access	Description / Value
0xF000	1	Index distance	UINT16	RO	Index distance between two modules = 0x10
	2	Maximum number of modules	UINT16	RO	Number of Modules = 1
	3	General configuration entries	UINT32	RO	Available Subindexes in General Configuration objects 0x8nn0 (Module Identification) as array of bits Bit 0 = 1 means 0x8nn0:Subindex 1 is available Bit 1 = 1 means 0x8nn0:Subindex 2 is available ... Bit 31 = 1 means 0x8nn0:Subindex 32 is available
	4	General information entries			Available Subindexes in General Information objects 0x9nn0 (Module Identification) as array of bits Bit 0 = 1 means 0x9nn0:Subindex 1 is available Bit 1 = 1 means 0x9nn0:Subindex 2 is available ... Bit 31 = 1 means 0x9nn0:Subindex 32 is available

E.2.2.2 Object 0xF100: Interbus Status

The Interbus status object (0xF100) shown in Table 81 is optional for an Interbus Slave.

Table 81: Interbus Slave Object 0xF100: Interbus Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF100	1	Interbus Status	UINT16	RO	Bit13: Interbus station error Bit 14: Interbus invalid data received all other bits are reserved
	16	TxPDO State	BOOL	RO	same value as 0x1800:07; will be set, if the Input Data is not valid (0xF100:01 > 0)

E.2.2.3 Object 0xF800: Interbus Parameter Settings

The Interbus Parameter Settings (0xF800) shown in Table 82 is optional for an Interbus Slave.

Table 82: Interbus Slave Object 0xF800: Interbus Parameter Settings

Index	Subindex	Name	Data Type	Access	Description / Value
0xF800	1	2MBit	BOOL	RW	0: Interbus runs with 500 kBit/s 1: Interbus runs with 2 Mbit/s
	2	Interbus ID	UINT8	RW	Interbus Device ID

E.2.2.4 Object 0xF801: Interbus Diagnosis

The Interbus Diagnosis object (0xF801) can be defined manufacturer specific.

E.3 PDO Mapping and Assign

E.3.1 Object 0x1600: RxPDO Mapping of the Interbus Slave

For the Interbus Slave one RxPDO Mapping object is reserved (see Table 83).

Table 83: Interbus Slave Objects 0x1600: RxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1600	1	Subindex 001	UINT32	0x700001ss (ss is the size of the Interbus Send Data of the Interbus Slave)
				If the Interbus Send Data size of is greater than 240 bits, only the first 240 bits are described with the first RxPDO mapping entry (0x700001E0), the following PDO mapping shall be used to described the missing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	Only if the Interbus Send Data size is greater than 240 bits
	...			

If the Interbus Slave has no Interbus Send Data this RxPDO Mapping object does not exist.

The RxPDO Mapping for the Interbus Slave is mandatory for the Interbus Slave Gateways.

E.3.2 Object 0x1A00: TxPDO Mapping of the Interbus Slave

For the Interbus Slave one TxPDO Mapping object is reserved (see Table 84).

Table 84: Interbus Slave Object 0x1A00: TxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1A00	1	Subindex 001	UINT32	0x600001ss (ss is the size of the Interbus Receive Data of the Interbus Slave)
				If the Interbus Receive Data size of is greater than 240 bits, only the first 240 bits are described with the first TxPDO mapping entry (0x600001E0), the following PDO mapping shall be used to described the missing size (while the missing size is greater than 240 bits, the next PDO mapping entry contains the value 0x000000E0, the last PDO mapping entry contains the missing size lower or equal than 240 bits)
	2	Subindex 002	UINT32	Only if the Interbus Receive Data size is greater than 240 bits
	...			

If the Interbus Slave has no Interbus Receive Data this TxPDO Mapping object does not exist.

The TxPDO Mapping for the Interbus Slave is mandatory for Interbus Slave Gateways.

E.3.3 Object 0x1A7F: Status TxPDO

The Status PDO is mandatory for the Interbus Slave.

If the mapped entries are not supported, the corresponding PDO mapping entries shall contain the align information.

Table 85: Interbus Slave Object 0x1A7F: TxPDO Mapping of Interbus Status

Subindex	Name	Data Type	Value
1	Subindex 001	UINT32	0x0000000F
2	Subindex 004	UINT32	0x18000701

E.3.4 Object 0x1C12: RxPDO Assign

The RxPDO of the Interbus Slave will be assigned first in the RxPDO Assign Object 0x1C12 (see Table 86) followed by optional vendor specific RxPDOs.

Table 86: Interbus Slave Object 0x1C12: RxPDO Assign

Subindex	Assigned RxPDO
1	0x1600
2	Vendor specific RxPDO if supported
3	...
...	

If the Interbus Slave does not have a RxPDO there is no gap in the RxPDO Assign Object.

E.3.5 Object 0x1C13: TxPDO Assign

The Status PDO will be assigned first, following the TxPDOs of the Interbus Slave and vendor specific TxPDOs as shown in Table 87.

Table 87: Interbus Slave Object 0x1C13: TxPDO Assign

Subindex	Assigned TxPDO
1	0x1A7F
2	0x1A00
3	Vendor specific TxPDO if supported
...	

If the Interbus Slave does not have a TxPDO there is no gap in the TxPDO Assign Object.

E.4 EtherCAT State Machine

Table 88 shows the behavior of the Interbus Slave in the different EtherCAT states.

Table 88: EtherCAT state behavior for the Interbus Slave fieldbus gateways

EtherCAT State	Behavior / Allowed Operations
INIT	Not connected to the Interbus
PRE-OP	Not connected to the Interbus
SAFE-OP	Connected to the Interbus the objects 0x6000 and 0x7000 will be created depending on the received Interbus Configuration The Interbus Send Data are in the safe state The TxPDO State and TxPDO Toggle will be updated
OP	The Interbus Send Data have process values as received via EtherCAT Output Data The EtherCAT Input Data will be updated with the received Interbus Data The TxPDO State and TxPDO Toggle will be updated

E.5 Synchronization

The Interbus Slave shall support the FreeRun Mode as defined in [6]

E.5.1 FreeRun Mode

In the FreeRun Mode the EtherCAT cycle is not synchronized with the Interbus cycle.

Annex F CAN Interface (Module Profile Number 5000)

F.1 General

The CAN Interface based on the modular device profile (fieldbus gateway) supports 1 module. This module contains a CAN TX message queue in the output data and a CAN RX message queue in the input data. The CAN Interface module can be integrated in the [CANopen Master](#) (see Annex G). In that case the CAN Interface shall be the first module of the CANopen Master.

F.2 Object Dictionary of the CAN Interface

The object dictionary of the device contains the object areas as shown in Table 89.

Table 89: CAN Interface device Object Dictionary Structure

Index	Object Dictionary Area
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x6000...0x6FFF	Input Area (CAN RX message queue)
0x7000...0x7FFF	Output Area (CAN TX message queue)
0x8000...0x8FFF	Configuration Area (configuration of the CAN Interface)
0xF000...0xFFFF	Device Area

In the following a Standard CAN message queue and an Extended CAN message queue is defined. The CAN Interface should supports either the Standard or the Extended CAN message queue during OP state. The message queue should be selected by writing the CAN Interface Settings object or by writing the RxPDO and TxPDO mapping object of the CAN Interface.

F.2.1 Object Areas of the CAN Interface (Module)

F.2.1.1 Output Data

The Output Data of the CAN Interface represent the TX message queue and the control information for the RX and the TX message queue. The Output Data is mandatory for a CAN Interface.

Standard CAN TX message queue

Table 90 describes the Object 0x7000 of the CAN interface. The maximum Subindex of object 0x7000 should be configured by writing the CAN Interface settings object or the RxPDO mapping object.

Table 90: CAN Interface Object 0x7000: CAN Tx Message Queue

Subindex	Name	Data Type	Access	Description / Value
1	TX Counter	UINT16	RO	Shall be incremented every time when new CAN TX messages to be sent are written in the process output data
2	RX Counter	UINT16	RO	Shall be incremented every time from the EtherCAT master's application to indicate that the received CAN RX messages are read and can be overwritten by the CAN Interface with new received RX messages
3	Number of TX Messages	UINT16	RO	Contains the number of CAN TX messages to be sent when TX Counter is incremented
4	TX Message 1	OCTET-STRING[12]	RO	1st CAN TX message to be sent when TX Counter was incremented: Bit 0-15: Transaction Number (the Transaction Action Number of the last CAN TX message sent shall be included in the input data) Bit 16-19: length of the CAN message (0-8) Bit 20: RTR-Bit Bit 21-31: CAN identifier Bit 32-95: CAN data
5	TX Message 2	OCTET-STRING[12]	RO	2nd CAN TX message to be sent when TX Counter was incremented
...				
255	TX Message 252	OCTET-STRING[12]	RO	252nd CAN TX message to be sent when TX Counter was incremented

Extended CAN TX message queue

Table 91 describes the Object 0x7001 of the CAN interface. The maximum Subindex of object 0x7001 should be configured by writing the CAN Interface settings object or the RxPDO mapping object.

The objects 0x7002-0x700F are vendor specific but shall be used according to the Modular Device Profile.

Table 91: CAN Interface Object 0x7001: Extended CAN TX Message Queue

Subindex	Name	Data Type	Access	Description / Value	
1	TX Counter	UINT16	RO	Shall be incremented every time when new CAN TX messages to be sent are written in the process output data	
2	RX Counter	UINT16	RO	Shall be incremented every time from the EtherCAT master's application to indicate that the received CAN RX messages are read and can be overwritten by the CAN Interface with new received RX messages	
3	Number of TX Messages	UINT16	RO	Contains the number of CAN TX messages to be sent when TX Counter is incremented	
4	TX Message 1	OCTET-STRING[16]	RO	1st CAN TX message to be sent when TX Counter was incremented:	
				Bit 0-15:	Transaction Number (the Transaction Action Number of the last CAN TX message sent shall be included in the input data)
				Bit 16-31:	length of the CAN message (0-8)
				Bit 32-60:	CAN identifier
				Bit 62:	RTR-Bit
				Bit 63:	0 (11 Bit identifier) 1 (29 Bit identifier)
				Bit 64-127:	CAN data
5	TX Message 2	OCTET-STRING[16]	RO	2nd CAN TX message to be sent when TX Counter was incremented	
...					
253	TX Message 250	OCTET-STRING[16]	RO	252nd CAN TX message to be sent when TX Counter was incremented	

F.2.1.2 Input Data

The Input Data of the CAN Interface represent the RX message queue and the status information for the RX and the TX message queue. The Input Data is mandatory for a CAN Interface.

Standard CAN RX message queue

Table 92 describes the Object 0x6000 of the CAN Interface. The maximum Subindex of object 0x6000 should be configured by writing the CAN Interface settings object or the TxPDO mapping object.

Table 92: CAN Interface Object 0x6000: Standard Rx Message Queue

Subindex	Name	Data Type	Access	Description / Value
1	TX Counter	UINT16	RO	Will be incremented every time from the CAN interface to indicate that the CAN TX messages are copied from the process data output to the local CAN send queue
2	RX Counter	UINT16	RO	Will be incremented from the CAN interface every time when new CAN RX messages are received and the Rx Counter (0x6000:02) is equal to the Rx Counter (0x7000:02) indicating that new CAN RX messages are written in the process input data
3	Number of RX Messages	UINT16	RO	Contains the number of CAN RX messages in the following input data when RX Counter is incremented
4	TX Transaction Number	UINT16	RO	Contains the Transaction Number of the last CAN TX message sent
5	RX Message 1	OCTET-STRING[10]	RO	1st new CAN RX message when RX Counter was incremented:
				Bit 0-3: Length of the CAN message (0-8)
				Bit 4: RTR-Bit
				Bit 5-15: CAN identifier
				Bit 16-79: CAN data
6	RX Message 2	OCTET-STRING[10]	RO	2nd new CAN RX message when RX Counter was incremented
...				
254	TX Message 250	OCTET-STRING[10]	RO	250th new CAN RX message when RX Counter was incremented

Extended CAN RX message queue

Table 93 describes the Object 0x6001 of the CAN Interface. The maximum Subindex of object 0x6001 should be configured by writing the CAN Interface settings object or the TxPDO mapping object.

The objects 0x6002-0x600F are vendor specific but shall be used according to the Modular Device Profile.

Table 93: CAN Interface Object 0x6001: Extended CAN Rx Message Queue

Subindex	Name	Data Type	Access	Description / Value
1	TX Counter	UINT16	RO	Will be incremented every time from the CAN interface to indicate that the CAN TX messages are copied from the process data output to the local CAN send queue
2	RX Counter	UINT16	RO	Will be incremented from the CAN interface every time when new CAN RX messages are received and the Rx Counter (0x6001:02) is equal to the Rx Counter (0x7001:02) indicating that new CAN RX messages are written in the process input data
3	Number of RX Messages	UINT16	RO	Contains the number of CAN RX messages in the following input data when RX Counter is incremented
4	TX Transaction Number	UINT16	RO	Contains the Transaction Number of the last CAN TX message sent
	RX Message 1	OCTET-STRING[14]	RO	1st new CAN RX message when RX Counter was incremented:
				Bit 0-15: Length of the CAN message (0-8)
				Bit 16-44: CAN identifier
				Bit 46: RTR-Bit
				Bit 47: 0 (11 Bit identifier), 1 (29 Bit identifier)
				Bit 48-111: CAN data
6	RX Message 2	OCTET-STRING[14]	RO	2nd new CAN RX message when RX Counter was incremented
...				
254 250	TX Message 250	OCTET-STRING[14]	RO	250th new CAN RX message when RX Counter was incremented

F.2.1.3 Configuration Data

The CAN Interface settings contain the information of the message format (Standard or Extended) and the size of the queues. These settings can be adapted alternatively by writing the PDO Mapping objects.

CAN Interface configuration

Table 94 shows the object 0x8000 of the CAN Interface.

Table 94: CAN Interface Object 0x8000: CAN Interface Configuration

Subindex	Name	Data Type	Access	Description / Value
1	Node Address	UINT16	RW	Shall be zero (to distinguish from the configuration of CANopen Slaves)
32	Flags	UINT16	RW	Bit 3: 0 = Standard CAN message queue 1 = Extended CAN message queue
33	RX Queue Size	UINT8	RW	Maximum number of CAN RX messages (1..250)
34	TX Queue Size	UINT8	RW	Maximum number of CAN TX messages (1..250)
35	Local RX Queue Size	UINT16	RW	Maximum number of CAN RX messages which could be stored in the CAN Interface
36	Local TX Queue Size	UINT16	RW	Maximum number of CAN TX messages which could be stored in the CAN Interface

The CAN Interface filter object contains the identifier areas which shall be queued in the CAN RX message queue. If this object is not set, the CAN Interface shall queue all received CAN messages in the CAN RX message queue.

CAN RX filter table

Table 95 shows the Object 0x8001 of the CAN Interface.

Table 95: CAN Interface Object 0x8001: Rx Filter Table

Subindex	Name	Data Type	Access	Description / Value	
1	Identifier Area 001	UINT64	RW	Bit 0-31:	First Identifier of this area to be queued
				Bit 32-63:	Last Identifier of this area to be queued
2	Identifier Area 002	UINT64	RW	Bit 0-31:	First Identifier of this area to be queued
				Bit 32-63:	Last Identifier of this area to be queued
..					
255	Identifier Area 255	UINT64	RW	Bit 0-31:	First Identifier of this area to be queued
				Bit 32-63:	Last Identifier of this area to be queued

F.2.2 Object Areas of the Device

The following objects of the Device Area are standardized for a CAN Interface (see Table 96). These objects shall be compatible to a CANopen Master because the CAN Interface could be integrated in the CANopen Master.

All other objects in the area 0xF000-0xFFFF are reserved for profile extensions.

Table 96: CAN Interface device Object Areas

Index	Object Name
0xF000	Modular device profile
0xF108	CAN Status
0xF800	CAN Bus parameter set

F.2.2.1 Object 0xF000: Modular device profile

The Modular device profile object (0xF000) shown in Table 97 is mandatory for the CAN Interface. If the CAN Interface is integrated in a CANopen Master, the object 0xF000 shall be defined as specified for the CANopen Master.

Table 97: CAN Interface Object 0xF000: Modular Device Profile

Subindex	Name	Data Type	Access	Description / Value
1	Index distance	UINT16	RO	Index distance between two modules = 0x10
2	Maximum number of modules	UINT16	RO	1
3	General configuration	UINT32	RO	Available entries in objects 0x8nn0 = 0x00000001

F.2.2.2 Object 0xF108: CAN Status

The CAN Status object (0xF108) as shown in Table 98 is optional for a CAN Interface.

Table 98: CAN Interface Object 0xF108: CAN Status

Subindex	Name	Data Type	Access	Description / Value
1	Bus Off	BOOLEAN	RO	TRUE: CAN-Controller is bus off
2	Warning Limit reached	BOOLEAN	RO	TRUE: Warning Limit of the CAN-Controller is reached
3	RX Overflow	BOOLEAN	RO	TRUE: CAN-Controller has a RX message overflow
4		BIT1	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=1)
5	TX Overflow	BOOLEAN	RO	TRUE: TX message overflow
6	Ack Error	BOOLEAN	RO	TRUE: CAN-Controller could not sent a message
7		BIT2	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=2)
9		BIT8	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=7)
17-32			RO	reserved for 16 bits of vendor specific status, the Subindex shall be the bit position as for Subindex 1-16
33	Rx Error Counter	UINT8	RO	RX Error Counter from CAN-Controller
34	Tx Error Counter	UINT8	RO	TX Error Counter from CAN-Controller

F.2.2.3 Object 0xF800: CAN Bus Parameter Set

The CAN Bus parameter set object (0xF800) is mandatory for a CAN Interface as shown in Table 99.

Table 99: CAN Interface Object 0xF800: CAN Bus Parameter Set

Subindex	Name	Data Type	Access	Description / Value
1		UINT8	RW	Reserved for CANopen Master (shall be 0x7F, if CANopen Master is not used)
2	Baudrate	UINT8	RW	Baudrate (0 = 1 MBaud, 1 = 800 kBaud, 2 = 500 kBaud, 3 = 250 kBaud, 4 = 125 kBaud, 5 = 100 kBaud, 6 = 50 kBaud, 7 = 20 kBaud, 8 = 10 kBaud, 255 = Baudrate defined in Busting Register (Subindex 5))
3		UINT16	RW	Reserved for CANopen Master (shall be 0x80, if CANopen Master is not used)
4		UINT32	RW	Reserved for CANopen Master (shall be 0, if CANopen Master is not used)
5	Busting Register	UINT32	RW	Bit 0-7: BT0-Register
				Bit 8-15: BT1-Register
				Bit 16-23: BT2-Register
				Bit 24-31: BT3-Register
6-n			RW	32 bytes of vendor specific configuration data, shall be zero, when accessing with CompleteAccess=TRUE, if not used

F.3 PDO Mapping and Assign

F.3.1 Object 0x1600: RxPDO Mapping of the CAN Interface

For the CAN Interface one RxPDO Mapping object is reserved. The RxPDO Mapping is mandatory for the CAN Interface.

Table 100: CAN Interface Object 0x1600: RxPDO Mapping (if Standard CAN msg queue selected)

Subindex	Name	Data Type	Description / Value
1	Subindex 001	UINT32	0x70000116 (TX Counter)
2	Subindex 002	UINT32	0x70000210 (RX Counter)
3	Subindex 003	UINT32	0x70000310 (Number of CAN TX Messages)
4	Subindex 004	UINT32	0x70000460 1. CAN TX Message
5	Subindex 005	UINT32	0x70000560 2. CAN TX Message
...			
253	Subindex 253	UINT32	0x7000FD60 250. CAN TX Message

Table 101: CAN Interface Object 0x1600: RxPDO Mapping (if Extended CAN msg queue selected)

Subindex	Name	Data Type	Description / Value
1	Subindex 001	UINT32	0x70010116 (TX Counter)
2	Subindex 002	UINT32	0x70010210 (RX Counter)
3	Subindex 003	UINT32	0x70010310 (Number of CAN TX Messages)
4	Subindex 004	UINT32	0x70010480 1. CAN TX Message
5	Subindex 005	UINT32	0x70010580 2. CAN TX Message
...			
253	Subindex 253	UINT32	0x7001FD80 250. CAN TX Message

F.3.2 Object 0x1A00: TxPDO Mapping of the CAN Interface

For the CAN Interface one TxPDO Mapping object is reserved. The TxPDO Mapping is mandatory for the CAN Interface.

Table 102: CAN Interface Object 0x1A00: TxPDO Mapping (if Standard CAN msg queue selected)

Subindex	Name	Data Type	Description / Value
1	Subindex 001	UINT32	0x60000116 (TX Counter)
2	Subindex 002	UINT32	0x60000210 (RX Counter)
3	Subindex 003	UINT32	0x60000310 (Number of CAN RX Messages)
4	Subindex 004	UINT32	0x60000410 (last TX Transaction Number)
5	Subindex 005	UINT32	0x60000550 1. CAN RX Message
6	Subindex 006	UINT32	0x60000650 2. CAN RX Message
...			
254	Subindex 254	UINT32	0x6000FE50 250. CAN TX Message

Table 103: CAN Interface Object 0x1600: TxPDO Mapping (if Extended CAN msg queue selected)

Subindex	Name	Data Type	Description / Value
1	Subindex 001	UINT32	0x60010116 (TX Counter)
2	Subindex 002	UINT32	0x60010210 (RX Counter)
3	Subindex 003	UINT32	0x60010310 (Number of CAN RX Messages)
4	Subindex 004	UINT32	0x60010410 (last TX Transaction Number)
5	Subindex 005	UINT32	0x60010570 1. CAN RX Message
6	Subindex 006	UINT32	0x60010670 2. CAN RX Message
...			
254	Subindex 253	UINT32	0x6001FE70 250. CAN RX Message

F.3.3 Object 0x1A85: TxPDO Mapping CAN Status

The CAN Status is defined in a TxPDO of the CAN Interface as shown in Table 104.

Table 104: CAN Interface Object 0x1A85: TxPDO Mapping of CAN Status

Subindex	Name	Data Type	Value
1	Bus Off	UINT32	0xF1080101
2	Warning Limit reached	UINT32	0xF1080201
3	RX Overflow	UINT32	0xF1080301
4	Align	UINT32	0x00000001
5	TX Overflow	UINT32	0xF1080501
6	Ack Error	UINT32	0xF1080601
7	Align	UINT32	0x0000000A
8-n	Vendor Specific Status	UINT32	
n+1	RX Error Counter	UINT32	0xF1082108
n+2	TX Error Counter	UINT32	0xF1082208

F.3.4 Object 0x1C12: RxPDO Assign

The RxPDO of the CAN Interface will be assigned in the RxPDO Assign Object 0x1C12 followed by optional vendor specific RxPDOs as shown in Table 105.

Table 105: CAN Interface Object 0x1C12: RxPDO Assign

Subindex	Assigned RxPDO
1	0x1600
2	Vendor specific RxPDO if supported
3	...
...	

If the CAN Interface is integrated in a CANopen Master, the RxPDO of the CAN-Interface shall be assigned first, followed by the RxPDOs for the CANopen Slaves and the vendor specific RxPDOs.

F.3.5 Object 0x1C13: TxPDO Assign

The TxPDO of the CAN Interface will be assigned in the TxPDO Assign Object 0x1C13 followed by optional vendor specific TxPDOs as shown in Table 106.

Table 106: CAN Interface Object 0x1C13: TxPDO Assign

Subindex	Assigned TxPDO
1	0x1A00
2	0x1A85
3	Vendor specific TxPDO if supported
...	...

If the CAN Interface is integrated in a CANopen Master, the TxPDO of the CAN-Interface shall be assigned first, followed by the TxPDOs for the CANopen Slaves, the TxPDOs for the CANopen Master and the vendor specific TxPDOs.

F.4 EtherCAT State Machine

Table 107 shows the behavior of the CAN Interface in the different EtherCAT states.

Table 107: CAN interface EtherCAT state behavior

EtherCAT State	Behavior / Allowed Operations
INIT	Not connected to the CAN-Bus, CAN-Controller in Reset If the backup parameter were stored, loading the stored configuration from non-volatile memory
PRE-OP	Connect to the CAN-Bus (Start CAN-Controller), when the Device Configuration Data object (0xF800) was downloaded completely, a stored configuration was loaded from non-volatile memory or the Detect Modules command was executed Create the objects 0x6000, 0x7000, 0x8000 when the object 0x8000 was downloaded completely or a stored configuration was loaded from non-volatile memory Optional: download of RX message filter object Optional: download of PDO Mapping Optional: download of PDO Assign
SAFE-OP	
OP	Sending CAN messages and receiving CAN messages

F.5 Device configuration

The Configuration Data objects (0x8000 and 0xF800) can be created in an CAN interface device in three different ways:

1. The objects should be created when they are downloaded completely the first time (with CompleteAccess=TRUE), starting with object 0xF800, followed by 0x8000
2. The objects should be created during start-up of the device in INIT state
3. The objects should be created when they were stored as backup parameter

The CAN Interface has to support at least one of the first two methods.

F.6 Synchronization

The CAN Interface shall support at the least one the synchronization modes as defined in [6]. This chapter describes the synchronization modes applied to the CAN Interface.

F.6.1 FreeRun Mode

In the FreeRun Mode the EtherCAT cycle is not synchronized with the CAN cycle.

F.6.2 Synchron Mode

In the Synchron Mode the EtherCAT cycle is synchronized with the CAN's sending cycle.

F.6.3 DC Mode

In DC Mode the EtherCAT cycle is synchronized with the CAN's sending cycle via Distributed Clocks.

Annex G CANopen Master (Module Profile Number 5100)

G.1 General

The CANopen Master gateway is based on the modular device profile (fieldbus gateway) with an object dictionary for each module (CANopen Slave). The CANopen Master gateway supports up to 127 modules (CANopen Slaves).

G.2 Object Dictionary of the CANopen Master

The object dictionary of the device contains the following object areas as shown in Table 108.

Table 108: CANopen Master device Object Dictionary Structure

Index	Object Dictionary Area
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x6000...0x6FFF	Input Area (TxPDOs of the CANopen slaves)
0x7000...0x7FFF	Output Area (RxPDOs of the CANopen slaves)
0x8000...0x8FFF	Configuration Area (Expected configuration of the CANopen slaves)
0x9000...0x9FFF	Information Area (Detected configuration of the CANopen slaves)
0xF000...0xFFFF	Device Area

G.2.1 Object Areas of the CANopen Slaves (Modules)

G.2.1.1 Output Data

The Output Data of the CANopen master represent the RxPDOs of the CANopen Slaves. Output Data is mandatory for a CANopen Master if CANopen slaves with RxPDOs are connected.

CAN Output Data Slave yyy

Table 109: CANopen Master Object 0x7nn0...0x7nn1: Output Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x7000	1	RxPDO 001	OCTET-STRING	RO	Represents RxPDO 1 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the RxPDO mapping is defined in the object dictionary of the CANopen Slave at index 0x1600
	2	RxPDO 002	OCTET-STRING	RO	Represents RxPDO 2 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the RxPDO mapping is defined in the object dictionary of the first CANopen Slave at index 0x1601
..					
0x7001	1	RxPDO 256	OCTET-STRING	RO	Represents RxPDO 256 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the RxPDO mapping is defined in the object dictionary of the first CANopen Slave at index 0x16FF
..					
	255	RxPDO 510	OCTET-STRING	RO	Represents RxPDO 510 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the RxPDO mapping is defined in the object dictionary of the first CANopen Slave at index 0x17FD
..					
0x7010	1	RxPDO 001	OCTET-STRING	RO	Represents RxPDO 1 of the second CANopen Slave (CANopen Node Address is defined in 0xF020:02), the RxPDO mapping is defined in the object dictionary of the second CANopen Slave at index 0x1600
...		...			

If a RxPDO does not exist, the related entry does not exist too and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

When no entry of an object is defined (e.g. object 0x7001 does not exist if the related CANopen Slave does not support the RxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

Depending on the Transmission type of the CANopen Slaves RxPDOs the CANopen Master will send the RxPDO CAN messages when the Output Data was written by the EtherCAT master.

The objects 0x7nn2-0x7nn3, 0x7nn6-0x7nn7, 0x7nnA-0x7nnB, 0x7xE-0x7nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x7nn4-0x7nn5, 0x7nn8-0x7nn9 and 0x7nnC-0x7nnD are reserved for profile extensions.

G.2.1.2 Input Data

The Input Data of the CANopen master represent the TxPDOs of the CANopen Slaves. Input Data is mandatory for a CANopen Master if CANopen slaves with TxPDOs are connected.

CAN Input Data Slave yyy

Table 110: CANopen Master Object 0x6nn0...0x6nn1: Input Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x6000	1	TxPDO 001	OCTET-STRING	RO	Represents TxPDO 1 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the TxPDO mapping is defined in the object dictionary of the first CANopen Slave at index 0x1A00
	2	TxPDO 002	OCTET-STRING	RO	Represents TxPDO 2 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the TxPDO mapping is defined in the object dictionary of the first CANopen Slave at index 0x1A01
..					
0x6001	1	TxPDO 256	OCTET-STRING	RO	Represents TxPDO 256 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the TxPDO mapping is defined in the object dictionary of the first CANopen Slave at index 0x1AFF
..					
	255	TxPDO 510	OCTET-STRING	RO	Represents TxPDO 510 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01), the TxPDO mapping is defined in the object dictionary of the first CANopen Slave at index 0x1BFD
..					
0x6010	1	TxPDO 001	OCTET-STRING	RO	Represents TxPDO 1 of the second CANopen Slave (CANopen Node Address is defined in 0xF020:02), the TxPDO mapping is defined in the object dictionary of the second CANopen Slave at index 0x1A00
...		...			

If a TxPDO does not exist, the related entry does not exist too and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

When no entry of an object is defined (e.g. object 0x6001 does not exist if the related CANopen Slave does not support the RxPDOs 256 to 510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The received TxPDOs of the CANopen slaves will be copied in the Input Data which is read by the EtherCAT master.

CAN TxPDO Toggle Slave yyy

The Objects 0x6nn4-0x6nn5 shown in Table 111 are optional for a CANopen Master.

Table 111: CANopen Master Objects 0x6nn4...0x6nn5: TxPDO Toggle

Index	Subindex	Name	Data Type	Access	Description / Value
0x6004	1	TxPDO 001	BOOLEAN	RO	The value is toggled every time when the TxPDO 1 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01) was received since the EtherCAT Input Data were updated the last time. This entry only make sense if the CANopen Master is synchronized with the EtherCAT Master
	2	TxPDO 002	BOOLEAN	RO	The value is toggled every time when the TxPDO 2 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01) was received since the EtherCAT Input Data were updated the last time. This entry only make sense if the CANopen Master is synchronized with the EtherCAT Master
..					
0x6005	1	TxPDO 256	BOOLEAN	RO	The value is toggled every time when the TxPDO 256 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01) was received since the EtherCAT Input Data were updated the last time. This entry only make sense if the CANopen Master is synchronized with the EtherCAT Master
..					
	255	TxPDO 510	BOOLEAN	RO	The value is toggled every time when the TxPDO 510 of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01) was received since the EtherCAT Input Data were updated the last time. This entry only make sense if the CANopen Master is synchronized with the EtherCAT Master
..					
0x6014	1	TxPDO 001	BOOLEAN	RO	The value is toggled every time when the TxPDO 1 of the second CANopen Slave (CANopen Node Address is defined in 0xF020:02) was received since the EtherCAT Input Data were updated the last time. This entry only make sense if the CANopen Master is synchronized with the EtherCAT Master
...		...			

If a TxPDO does not exist, the related entry does not exist too and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

The objects 0x6nn2-0x6nn3, 0x6nn6-0x6nn7, 0x6nnA-0x6nnB and 0x6nnE-0x6nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x6nn8-0x6nn9 and 0x6nnC-0x6nnD are reserved for profile extensions.

G.2.1.3 Configuration Data

The Configuration Data contain the CANopen communication parameter the CANopen Master needs to establish a CANopen connection to the CANopen Slave. The Configuration Data is mandatory for a CANopen Master.

CAN Communication Parameter Slave yyy

The objects 0x8nn0 shown in Table 112 are mandatory for a CANopen Master.

The CANopen Master will automatically update the object 0xF020 when the objects 0x8nn0 are written by the EtherCAT master.

Table 112: CANopen Master Object 0x8nn0: CAN Communication Parameter

Index	Subindex	Name	Data Type	Access	Description / Value
0x8000	1	Node Address	UINT16	RW	Node Address of the first CANopen Slave (same value as 0xF020:01)
	4	Device Type	UINT32	RW	Expected Device Type (Object 0x1000) of the first CANopen Slave, 0: value will not be checked during CANopen boot up
	5	Vendor ID	UINT32	RW	Expected Vendor ID (Entry 0x1018:01) of the first CANopen Slave, 0: value will not be checked during CANopen boot up
	6	Product Code	UINT32	RW	Expected Product Code (Entry 0x1018:02) of the first CANopen Slave, 0: value will not be checked during CANopen boot up
	7	Revision	UINT32	RW	Expected Revision (Entry 0x1018:03) of the first CANopen Slave, 0: value will not be checked during CANopen boot up
	8	Serial Number	UINT32	RW	Expected Serial Number (Entry 0x1018:04) of the first CANopen Slave, 0: value will not be checked during CANopen boot up
	29	Network Flags	UINT16	RW	Network Flags to route vendor specific services over CANopen, shall be 0, if not used
	30	Network Port	UINT16	RW	Network Port to route vendor specific services over CANopen, shall be 0, if not used
	31	Network Segment Address	OCTET-STRING[6]	RW	Network Segment Address to route vendor specific services over CANopen, shall be 0, if not used
	32	Flags	UINT16	RW	Bit 0: Download of PDO communication parameter during CANopen boot up disabled
					Bit 1: Try using Heartbeat instead of Guarding with the CANopen Slave
					Bit 2-7: Reserved
					Bit 8-15: Vendor specific
	33	Guarding Time	UINT16	RW	Guarding Time (Object 0x100C) set to the first CANopen Slave, 0: no guarding
	34	Life Time Factor	UINT16	RW	Life Time Factor (Object 0x100D) set to the first CANopen Slave, 0: no guarding
	35	SDO-Timeout	UINT16	RW	SDO-Timeout for the first CANopen Slave in ms (time how long the CANopen Master waits for a SDO response of the CANopen Slave)
	36	Boot-Timeout	UINT16	RW	Boot-Timeout for the first CANopen Slave (time how long the CANopen Master will wait after sending the Reset Communication NMT until the first SDO request will be sent)
	37-n			RW	32 bytes of vendor specific configuration data, shall be zero, when accessing with CompleteAccess=TRUE, if not used
..					
0x8010	1	Node Address	UINT16	RW	Node Address of the second CANopen Slave (same value as 0xF020:02)
...		...			

CAN SDO Init Cmds Slave yyy

The SDO Init Cmds are SDO Download services which shall be sent to the CANopen Slave every time before starting this CANopen Slave. The objects 0x8nn3 are optional for a CANopen master.

Table 113: CANopen Master Object 0x8nn3: SDO Init Cmds

Index	Subindex	Name	Data Type	Access	Description / Value
0x8003	1	Subindex 001	OCTET-STRING[n]	RW	First SDO Init Cmd of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
					Byte 0-1: Index of the SDO Download service
					Byte 2: Subindex of the SDO Download service
					Byte 3-4: Length of the SDO Download service
					Byte 5-(n-1): Data of the SDO download service
	2	Subindex 002	OCTET-STRING[n]	RW	Second SDO Init Cmd of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
	...				
	255	Subindex 255	OCTET-STRING[n]	RW	255th SDO Init Cmd of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
	...				
0x8013	1	Subindex 001	OCTET-STRING[n]	RW	First SDO Init Cmd of the second CANopen Slave (CANopen Node Address is defined in 0xF020:02)
	...				

CAN TxPDO Configuration Slave yyy

Table 114: CANopen Master Object 0x8nn6...0x8nn7: TxPDO Configuration

Index	Subindex	Name	Data Type	Access	Description / Value
0x8006	1	TxPDO 001	OCTET-STRING[12]	RW	TxPDO 1 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
					Byte 0-3: COB-ID of the TxPDO (Entry 0x1800:01 of the first CANopen Slave)
					Byte 4: Transmission Type of the TxPDO (Entry 0x1800:02 of the first CANopen Slave)
					Byte 5: Length of the TxPDO (1-8)
					Byte 6-7: Inhibit Time of the TxPDO (Entry 0x1800:03 of the first CANopen Slave)
					Byte 8-9: Event Time of the TxPDO (Entry 0x1800:05 of the first CANopen Slave)
					Byte 10-11: Flags Bit 0: PDO-Toggle (Entry 0x6004:01) shall be mapped in TxPDO 131 (object 0x1A82)
					Bit 1-7: Reserved
					Bit 8-15: Vendor specific
	2	TxPDO 002	OCTET-STRING[12]	RW	TxPDO 2 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
	...				
0x8007	1	TxPDO 256	OCTET-STRING[12]	RW	TxPDO 256 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
	...				
	255	TxPDO 510	OCTET-STRING[12]	RW	TxPDO 510 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
0x8016	1	TxPDO 001	OCTET-STRING[12]	RW	Configuration of the first TxPDO of the second CANopen Slave (CANopen Node Address is defined in 0xF020:02)
	...				

If a TxPDO does not exist, the related entry exists as NULL-Entry if entries of this object follow. A SDO Upload or SDO Download service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing). A SDO Information service with the related subindex addressed shall return an entry description with DataType=0 and BitLength=80 (see chapter SDO Complete Access in [6]). The definition of NULL-Entries is needed that a SDO Download with CompleteAccess=TRUE can be supported.

If a TxPDO does not exist, the related Subindex does not exist if no entries of this object follow. A SDO Upload, SDO Download or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x8007 does not exist if the related CANopen Slave does not support the TxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

CAN RxPDO Configuration Slave yyy

Table 115: CANopen Master Object 0x8nn8...0x8nn9: RxPDO Configuration

Index	Subindex	Name	Data Type	Access	Description / Value
0x8008	1	RxPDO 001	OCTET-STRING[12]	RW	RxPDO 1 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
					Byte 0-3: COB-ID of the RxPDO (Entry 0x1400:01 of the first CANopen Slave)
					Byte 4: Transmission Type of the RxPDO (Entry 0x1400:02 of the first CANopen Slave)
					Byte 5: Length of the RxPDO (1-8)
					Byte 6-7: Inhibit Time of the RxPDO (Entry 0x1400:03 of the first CANopen Slave)
					Byte 8-9: Event Time of the RxPDO (Entry 0x1400:05 of the first CANopen Slave)
					Byte 10-11: Flags Bit 0-7: Reserved Bit 8-15: Vendor specific
	2	RxPDO 002	OCTET-STRING[12]	RW	RxPDO 2 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
	...				
0x8009	1	RxPDO 256	OCTET-STRING[12]	RW	RxPDO 256 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
	...				
	255	RxPDO 510	OCTET-STRING[12]	RW	RxPDO 510 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF020:01)
0x8018	1	RxPDO 001	OCTET-STRING[12]	RW	Configuration of the first RxPDO of the second CANopen Slave (CANopen Node Address is defined in 0xF020:02)
	...				

If a RxPDO does not exist, the related entry exists as NULL-Entry if entries of this object follows. A SDO Upload or SDO Download service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing). A SDO Information service with the related subindex addressed shall return an entry description with DataType=0 and BitLength=80.

If a RxPDO does not exist, the related does not exist if no entries of this object follows. A SDO Upload, SDO Download or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x8009 does not exist if the related CANopen Slave does not support the RxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The objects 0x8nn1-0x8nn4 are vendor specific but shall be used according to the Modular Device Profile. The objects 0x8nnA-0x8nnF are reserved for profile extensions.

G.2.1.4 Information Data

The Information Data contain the information about the connected CANopen slaves. It will be available when the Detect Modules Command (see G.2.2.2) is executed. The Information Data is optional for a CANopen Master because all the information are available for the EtherCAT master by reading the objects directly from the CANopen Slave.

Detected CANopen Identification Slave yyy

Table 116: CANopen Master Object 0x9nn0: Detected CANopen Identification

Index	Subindex	Name	Data Type	Access	Description / Value
0x9000	1	Node Address	UINT16	RO	Node Address of the first CANopen Slave found (same value as 0xF040:01)
	3	Manufacturer Device Name	VISIBLE-STRING	RO	Manufacturer Device Name (Object 0x1008) of the first CANopen Slave found
	4	Device Type	UINT32	RO	Device Type (Object 0x1000) of the first CANopen Slave found
	5	Vendor ID	UINT32	RO	Vendor ID (Entry 0x1018:01) of the first CANopen Slave found
	6	Product Code	UINT32	RO	Product Code (Entry 0x1018:02) of the first CANopen Slave found
	7	Revision	UINT32	RO	Revision (Entry 0x1018:03) of the first CANopen Slave found
	8	Serial Number	UINT32	RO	Serial Number (Entry 0x1018:04) of the first CANopen Slave found
..					
0x9010	1	Node Address	UINT16	RO	Node Address of the second CANopen Slave found (same value as 0xF040:02)
...		...			
0x9000	1	Node Address	UINT16	RO	Node Address of the first CANopen Slave found (same value as 0xF040:01)

When accessing the object with CompleteAccess=TRUE the values of the entries 1,2,4,5,6,7,8 will be packed one after another without spaces.

Detected CAN TxPDO Configuration Slave yyy

Table 117: CANopen Master Object 0x9nn6...0x9nn7: Detected CAN TxPDO Configuration

Index	Subindex	Name	Data Type	Access	Description / Value
0x9006	1	TxPDO 001	OCTET-STRING[12]	RO	TxPDO 1 Configuration of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01) Structure like object 0x8006 (see Table 114)
	2	TxPDO 002	OCTET-STRING[12]	RO	TxPDO 2 Configuration of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
0x9007	1	TxPDO 256	OCTET-STRING[12]	RO	TxPDO 256 Configuration of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
	255	TxPDO 510	OCTET-STRING[12]	RO	TxPDO 510 Configuration of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
0x9016	1	TxPDO 001	OCTET-STRING[12]	RO	Configuration of the first TxPDO of the second CANopen Slave found (CANopen Node Address is defined in 0xF040:02)
	...				

If a TxPDO does not exist, the related entry exists as NULL-Entry if entries of this object follow. A SDO Upload service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing). A SDO Information service with the related subindex addressed shall return an entry description with DataType=0 and BitLength=80.

If a TxPDO does not exist, the related does not exist if no entries of this object follow. A SDO Upload or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x9007 does not exist if the related CANopen Slave does not support the TxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

Detected CAN RxPDO Configuration Slave yyy

Table 118: CANopen Master Object 0x9nn8...0x9nn9: Detected CAN RxPDO Configuration

Index	Subindex	Name	Data Type	Access	Description / Value
0x9008	1	RxPDO 001	OCTET-STRING[12]	RO	RxPDO 1 Configuration of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01) Structure like object 0x8008 (see Table 115)
	2	RxPDO 002	OCTET-STRING[12]	RO	RxPDO 2 Configuration of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
0x9009	1	RxPDO 256	OCTET-STRING[12]	RO	RxPDO 256 Configuration of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
	255	RxPDO 510	OCTET-STRING[12]	RO	RxPDO 510 Configuration of the first CANopen Slave (CANopen Node Address is defined in 0xF040:01)
	...				
0x9018	1	RxPDO 001	OCTET-STRING[12]	RO	Configuration of the first RxPDO of the second CANopen Slave found (CANopen Node Address is defined in 0xF040:02)
	...				

If a RxPDO does not exist, the related entry exists as NULL-Entry if entries of this object follows. A SDO Upload service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing). A SDO Information service with the related subindex addressed shall return an entry description with DataType=0 and BitLength=80.

If a RxPDO does not exist, the related does not exist if no entries of this object follows. A SDO Upload or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x9009 does not exist if the related CANopen Slave does not support the RxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

Detected CAN TxPDO Mapping Slave yyy

Table 119: CANopen Master Object 0x9nnA...0x9nnB: Detected CAN TxPDO Mapping

Index	Subindex	Name	Data Type	Access	Description / Value
0x900A	1	TxPDO 001	OCTET-STRING[n]	RO	TxPDO 1 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
					Byte 0-3: 1st mapping entry of the TxPDO (Entry 0x1A00:01 of the first CANopen Slave)
					Byte 4-7: 2nd mapping entry of the TxPDO (Entry 0x1A00:02 of the first CANopen Slave)
					...
	2	TxPDO 002	OCTET-STRING[n]	RO	TxPDO 2 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
0x900B	1	TxPDO 256	OCTET-STRING[n]	RO	TxPDO 256 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
	255	TxPDO 510	OCTET-STRING[n]	RO	TxPDO 510 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
0x901A	1	TxPDO 001	OCTET-STRING[n]	RO	Mapping of the first TxPDO of the second CANopen Slave found (CANopen Node Address is defined in 0xF040:02)
	...				

If a TxPDO does not exist, the related does not exist. A SDO Upload or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. Object 0x900B does not exist if the related CANopen Slave does not support the TxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

Detected CAN RxPDO Mapping Slave yyy

Table 120: CANopen Master Object 0x9nnC...0x9nnD: Detected RxPDO Mapping

Index	Subindex	Name	Data Type	Access	Description / Value
0x900C	1	RxPDO 001	OCTET-STRING[n]	RO	RxPDO 1 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
					Byte 0-3: 1st mapping entry of the RxPDO (Entry 0x1600:01 of the first CANopen Slave)
					Byte 4-7: 2nd mapping entry of the RxPDO (Entry 0x1600:02 of the first CANopen Slave)
					...
	2	RxPDO 002	OCTET-STRING[n]	RO	RxPDO 2 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
0x900D	1	RxPDO 256	OCTET-STRING[n]	RO	RxPDO 256 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
	255	RxPDO 510	OCTET-STRING[n]	RO	RxPDO 510 Mapping of the first CANopen Slave found (CANopen Node Address is defined in 0xF040:01)
	...				
0x901C	1	RxPDO 001	OCTET-STRING[n]	RO	Mapping of the first RxPDO of the second CANopen Slave found (CANopen Node Address is defined in 0xF040:02)
	...				

If a RxPDO does not exist, the related does not exist. A SDO Upload or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. Object 0x900E does not exist if the related CANopen Slave does not support the RxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The objects 0x9nn1-0x9nn4 are vendor specific but shall be used according to the Modular Device Profile. The objects 0x9nn5 and 0x9nnE-0x9nnF are reserved for profile extensions.

G.2.1.5 Diagnosis Data

No standardized Diagnosis Data defined.

G.2.2 Object Areas of the Device

The following objects as shown in Table 121 of the Device Area are standardized for a CANopen Master. All other objects in the area 0xF000...0xFFFF are reserved for profile extensions.

Table 121: CANopen Master Device Object Areas

Index	Object Name
0xF000	Modular device profile
0xF002	Detect modules
0xF010	Module profile list
0xF020	Configured address list
0xF040	Detected address list
0xF108	CAN Status
0xF800	CAN Bus parameter set

G.2.2.1 Object 0xF000: Modular Device Profile

The Modular device profile object (0xF000) as shown in Table 122 is mandatory for a CANopen Master.

Table 122: CANopen Master Object 0xF000: Modular Device Profile

Index	Subindex	Name	Data Type	Access	Description / Value
0xF000	1	Index distance	UINT16	RO	Index distance between two modules = 0x10
	2	Maximum number of modules	UINT16	RO	Maximum number of CANopen Slaves connected to the CANopen Master = 127
	3	General configuration	UINT32	RO	Available entries in objects 0x8nn0 = 0x0000007D
	4	General information	UINT32	RO	Available entries in objects 0x9nn0 = 0x0000007D (if information data supported) or 0x00000000 (if information data not supported)

G.2.2.2 Object 0xF002: Detect Modules Command

The Detect modules Command object (0xF002) as shown in Table 123 is optional for a CANopen Master.

Table 123: CANopen Master Object 0xF002: Detect Modules Command

Index	Subindex	Name	Data Type	Access	Description / Value
0xF002	1	Command Request	OCTET-STRING[1]	RW	Byte 0: Baudrate (0 = 1 MBaud, 1 = 800 kBaud, 2 = 500 kBaud, 3 = 250 kBaud, 4 = 125 kBaud, 5 = 100 kBaud, 6 = 50 kBaud, 7 = 20 kBaud, 8 = 10 kBaud) When this subindex is written from the EtherCAT Master, the CANopen Master shall start its CAN Controller with the requested baudrate
	2	Command Status	UINT8	RO	1: command is finished, no error 3: command is finished, error 255: command is executing
	3	Command Response	OCTET-STRING	RO	Byte 0: like Subindex 2 Byte 1: always 0 Byte 2-3: 0: no error, > 0: vendor specific error code Byte 4: Node Address of the first CANopen Slave found Byte 5-8: Vendor ID (0x1018:01) of the first CANopen Slave found Byte 9-12: Product Code (0x1018:02) of the first CANopen Slave found Byte 13: Node Address of the second CANopen Slave found Byte 14-17: Vendor ID (0x1018:01) of the second CANopen Slave found Byte 18-21: Product Code (0x1018:02) of the second CANopen Slave found ...

G.2.2.3 Object 0xF010: Module Profile List

The Module profile list object (0xF010) as shown in Table 124 is optional for a CANopen Master.

Table 124: CANopen Master Object 0xF010: Module Profile List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF010	1	Subindex 001	UINT32	RO	Device Type (Object 0x1000) of the first CANopen Slave configured (CANopen Node Address is defined in 0xF020:01)
	2	Subindex 002	UINT32	RO	Device Type (Object 0x1000) of the second CANopen Slave configured (CANopen Node Address is defined in 0xF020:02)
...					
	127	Subindex 127	UINT32	RO	Device Type (Object 0x1000) of the hundred twenty seventh CANopen Slave configured (CANopen Node Address is defined in 0xF020:7F)

G.2.2.4 Object 0xF020: Configured Address List

The Configured address list object (0xF020) as shown in Table 125 is mandatory for a CANopen Master.

Table 125: CANopen Master Object 0xF020: Configured Address List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF020	1	Subindex 001	UINT16	RO	CANopen Node Address of the first CANopen Slave configured
	2	Subindex 002	UINT16	RO	CANopen Node Address of the second CANopen Slave configured
...					
	127	Subindex 127	UINT16	RO	CANopen Node Address of the hundred twenty seventh CANopen Slave configured

G.2.2.5 Object 0xF040: Detected Address List

The Detected address list object (0xF040) as shown in Table 126 is optional for a CANopen Master.

Table 126: CANopen Master Object 0xF040: Detected Address List

Index	Subindex	Name	Data Type	Access	Description / Value
0xF040	1	Subindex 001	UINT16	RO	CANopen Node Address of the first CANopen Slave found
	2	SubIndex 002	UINT16	RO	CANopen Node Address of the second CANopen Slave found
...					
	127	Subindex 127	UINT16	RO	CANopen Node Address of the hundred twenty seventh CANopen Slave found

G.2.2.6 Object 0xF101: CANopen Master Status

The CAN Status object (0xF101) as shown in Table 127 is optional for a CANopen Master.

Table 127: CANopen Master Object 0xF101: CANopen Master Status

Index	SubIndex	Name	Data Type	Access	Description
0xF101	12	SYNC Toggle	BOOLEAN	RO	toggles with each transmission of the SYNC message
	15	PDO Toggle	BOOLEAN	RO	The bit toggles if the EtherCAT input data has been updated since the previous EtherCAT input update
	16	PDO State	BOOLEAN	RO	This bit is set if at least one configured CANopen slave has a node state that is not equal to 0
	17	Cycle Count	UINT16	RO	This counter is incremented after each CAN cycle (if at least one CANopen slave has been configured)
	18	Slave Status Counter	UINT8	RO	This byte contains the number of
	20	Cycle Time	UINT16	RO	This entry contains the required cycle time of the CAN cycle in 1/9 μ s

G.2.2.7 Object 0xF102: Node state list

The Node state list object (0xF102) as shown in Table 128 is optional for a CANopen Master.

Table 128: CANopen Master Object 0xF102: Node state list

Index	SubIndex	Name	Data Type	Access	Description
0xF102	1	Node State Slave 1	UINT8	RO	CANopen Node state of the first CANopen Slave configured
					0 No Error
					1 CANopen slave has not been restarted following an error, because entry 0x8yy0:27 has been configured for manual restart or because the CANopen slave was stopped with AoE
					2 CANopen slave does not respond
					3 reserved
					4 Length of the data at a StartUp SDO upload is incorrect or StartUp SDO download has failed
					5 Value of the data at a StartUp SDO upload is incorrect
					6...7 reserved
					8 CANopen slave is in boot-up (StartUp SDOs are being transmitted, so far no error)
					9..10 reserved
					11 CAN controller is in bus-off
					12 CANopen slave has left OPERATIONAL
					13 reserved
					14 Guarding has not toggled
					15..17 reserved
					18 CANopen slave was started, all CAN TxPDOs were received, but no EtherCAT process data has been exchanged yet
					19 reserved
					20 CAN TxPDO with incorrect length received
					21 reserved
					22 synchronous or event-timer-triggered CAN TxPDO was not received in time
					23 at least one CAN TxPDO has not yet been received after the Start Node
					24 TX FIFO overflow (e.g. if no CAN acknowledge is detected)
					25...39 reserved
					40 CAN TxPDO with type 1 transmission was not received in this CAN cycle
	2	Node State Slave 2	UINT8	RO	CANopen Node state of the second CANopen Slave configured
...					
	127	Node State Slave 127	UINT8	RO	CANopen Node state of the hundred twenty seventh CANopen Slave configured

G.2.2.8 Object 0xF108: CAN Status

The CAN Status object (0xF108) as shown in Table 129 is optional for a CANopen Master.

Table 129: CANopen Master Object 0xF108: CAN Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF108	1	Bus Off	BOOLEAN	RO	TRUE: CAN-Controller is bus off
	2	Warning Limit reached	BOOLEAN	RO	TRUE: Warning Limit of the CAN-Controller is reached
...	3	RX Overflow	BOOLEAN	RO	TRUE: CAN-Controller has a RX message overflow
	4		BIT1	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=1)
	5	TX Overflow	BOOLEAN	RO	TRUE: TX message overflow
	6	Ack Error	BOOLEAN	RO	TRUE: CAN-Controller could not sent a message
	7		BIT2	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=2)
	9		BIT8	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=7)
	17-32			RO	Reserved for 16 bits of vendor specific status, the Subindex shall be the bit position as for Subindex 1-16
	33	Rx Error Counter	UINT8	RO	RX Error Counter from CAN-Controller
	34	Tx Error Counter	UINT8	RO	TX Error Counter from CAN-Controller

G.2.2.9 Object 0xF800: CAN Bus Parameter Set

The CAN Bus parameter set object (0xF800) as shown in Table 130 is mandatory for a CANopen Master.

Table 130: CANopen Master Object 0xF800: CAN Bus Parameter Set

Index	Subindex	Name	Data Type	Access	Description / Value	
0xF800	1	CANopen Node Address, CANopen Master	UINT8	RW	CANopen Node Address of the CANopen Master (needed for the CANopen Master's Heartbeat message)	
	2	Baudrate	UINT8	RW	Baudrate (0 = 1 MBaud, 1 = 800 kBaud, 2 = 500 kBaud, 3 = 250 kBaud, 4 = 125 kBaud, 5 = 100 kBaud, 6 = 50 kBaud, 7 = 20 kBaud, 8 = 10 kBaud)	
	3	COB ID SYNC	UINT16	RW	COB-ID for the SYNC message	
	4	SYNC Cycle Time	UINT32	RW	Time distance between two SYNC messages in μ s	
	5	Bustiming Register	UINT32	RW	Bit 0-7:	BT0-Register
					Bit 8-15:	BT1-Register
					Bit 16-23:	BT2-Register
					Bit 24-31:	BT3-Register
	6-n			RW	32 bytes of vendor specific configuration data, shall be zero, when accessing with CompleteAccess=TRUE, if not used	

G.3 PDO Mapping and Assign

G.3.1 Objects 0x16nn: RxPDO Mapping of the CANopen Slaves

For each CANopen Slave one RxPDO Mapping object is reserved as shown in Table 131.

Table 131: CANopen Master Objects 0x1600...0x167E: RxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1600	1	Subindex 001	UINT32	0x7000ppss (pp depends on the related RxPDO of the first configured CANopen Slave (CANopen Node Address in entry 0xF020:01), ss is the size of this RxPDO in bits, starting with the first RxPDO this Slave)
	2	Subindex 002	UINT32	0x7000ppss (pp depends on the related RxPDO of the first configured CANopen Slave (CANopen Node Address in entry 0xF020:01), ss is the size of this RxPDO in bits, starting with the first RxPDO this Slave)
	...			
0x1601	1	Subindex 001	UINT32	0x7010ppss (pp depends on the related RxPDO of the second configured CANopen Slave (CANopen Node Address in entry 0xF020:02), ss is the size of this RxPDO in bits, starting with the first RxPDO this Slave)
	...			

If a CANopen Slave has no RxPDOs the related RxPDO Mapping object does not exist.

The RxPDO Mapping for each CANopen Slave is mandatory for a CANopen Master.

G.3.2 Objects 0x1Ann: TxPDO Mapping of CANopen Slaves

For each CANopen Slave one TxPDO Mapping object is reserved as shown in Table 132.

Table 132: CANopen Master Objects 0x1A00...0x1A7E: TxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1A00	1	Subindex 001	UINT32	0x7000ppss (pp depends on the related RxPDO of the first configured CANopen Slave (CANopen Node Address in entry 0xF020:01), ss is the size of this RxPDO in bits, starting with the first RxPDO this Slave)
	2	Subindex 002	UINT32	0x7000ppss (pp depends on the related RxPDO of the first configured CANopen Slave (CANopen Node Address in entry 0xF020:01), ss is the size of this RxPDO in bits, starting with the first RxPDO this Slave)
	...			
0x1A01	1	Subindex 001	UINT32	0x7010ppss (pp depends on the related RxPDO of the second configured CANopen Slave (CANopen Node Address in entry 0xF020:02), ss is the size of this RxPDO in bits, starting with the first RxPDO this Slave)
	...			

If a CANopen Slave has no TxPDOs the related TxPDO Mapping object does not exist.

The TxPDO Mapping for each CANopen Slave is mandatory for a CANopen Master.

G.3.3 Object 0x1A81: TxPDO Mapping of CANopen State

The PDO State as shown in Table 133 is used to signal if the communication between the CANopen Master and the CANopen Slaves is established. If a CANopen Slave has a related TxPDO the TxPDO State is defined, if a CANopen Slave only has a related RxPDO the RxPDO State is defined.

Table 133: CANopen State

Description	Index	Sub-index	Data Type	Value
PDO State of 1st CANopen Slave configured	TxPDO 1 exists: 0x1800	7	BOOLEAN	0: CANopen Slave is OPERATIONAL 1: CANopen Slave is not OPERATIONAL
PDO State of 2nd CANopen Slave configured	TxPDO 2 exists: 0x1801 TxPDO 2 do not exist: 0x1401	7	BOOLEAN	0: CANopen Slave is OPERATIONAL 1: CANopen Slave is not OPERATIONAL
...				

The CANopen States of the configured CANopen Slaves are mapped to TxPDO 130 (TxPDO Mapping object 0x1A81) as shown in Table 134.

Table 134: CANopen Master Object 0x1A81: TxPDO Mapping of CANopen States

Subindex	Name	Data Type	Value
1	PDO State 1st CANopen Slave configured	UINT32	0x18000701 (if 0x1A00 exists) 0x14000701 (if 0x1A00 do not exist)
2	PDO State 2nd CANopen Slave configured	UINT32	0x18010701 (if 0x1A01 exists) 0x14010701 (if 0x1A01 do not exist)
...			

The CANopen State is mandatory for CANopen Master.

G.3.4 Object 0x1A82: TxPDO Mapping of CAN Status

The CAN Status is defined in a TxPDO of the CANopen Master as shown in Table 135.

Table 135: CANopen Master Object 0x1A82: TxPDO Mapping of CAN Status

Subindex	Name	Data Type	Value
1	Bus Off	UINT32	0xF1080101
2	Warning Limit reached	UINT32	0xF1080201
3	RX Overflow	UINT32	0xF1080301
4	Align	UINT32	0x00000001
5	TX Overflow	UINT32	0xF1080501
6	Ack Error	UINT32	0xF1080601
7	Align	UINT32	0x0000000A
8-n	Vendor Specific Status	UINT32	
n+1	RX Error Counter	UINT32	0xF1082108
n+2	TX Error Counter	UINT32	0xF1082208

G.3.5 Object 0x1A88...0x1A8F: TxPDO Mapping of PDO Toggle

For the CANopen Master the standard PDO Toggle cannot be used because the process data of a CANopen Slave may be sent with different transmission types. So a PDO Toggle for every EtherCAT-PDO is not enough, a PDO-Toggle is needed for every CANopen-PDO. The CANopen PDO-Toggle for TxPDOs was described above in the chapter G.2.1.2. The RxPDO Toggle is not defined for the CANopen Master.

In the object 0x8nn6 (see chapter G.2.1.3) is defined which CANopen TxPDO Toggle entries shall be transmitted via EtherCAT in the TxPDOs 136-143 (TxPDO Mapping objects 0x1A88-0x1A8F).

Table 136: CANopen Master Objects 0x1A88...0x1A8F: TxPDO Mapping of CANopen TxPDO Toggle

Index	Subindex	Name	Data Type	Value
0x1A88	1	1st mapped CANopen TxPDO Toggle	UINT32	0x6nn4pp01 (nn depends on the related CANopen Slave, pp on the related TxPDO of this Slave, starting with the first TxPDO of the first CANopen Slave configured which has the TxPDO Toggle enabled (in entry 0x8nn1:pp, bit 10.0))
0x1A88	2	2nd mapped CANopen TxPDO Toggle	UINT32	0x6nn4pp01 (nn depends on the related CANopen Slave, pp on the related TxPDO of this Slave, starting with the first TxPDO of the first CANopen Slave configured which has the TxPDO Toggle enabled (in entry 0x8nn1:pp, bit 10.0))
...	...			
0x1A89	1	256th mapped CANopen TxPDO Toggle	UINT32	0x6nn4pp01 (nn depends on the related CANopen Slave, pp on the related TxPDO of this Slave, starting with the first TxPDO of the first CANopen Slave configured which has the TxPDO Toggle enabled (in entry 0x8nn1:pp, bit 10.0))
...	...			

The PDO Toggle is optional for the CANopen Master.

G.3.6 Object 0x1A83: TxPDO Mapping of CANopen Node State

The CANopen Node States of the configured CANopen Slaves are mapped to TxPDO 132 (TxPDO Mapping object 0x1A83) as shown in Table 137.

Table 137: CANopen Master object 0x1A83: TxPDO Mapping for Node States

SubIndex	Description	Data Type	Value
1	Node State 1st CANopen Slave configured	UINT32	0xF1020108
2	Node State 2nd CANopen Slave configured	UINT32	0xF1020208
...			

The CANopen Node State is Optional for CANopen Master.

G.3.7 Object 0x1A84: TxPDO Mapping of CANopen Master Status

The CANopen Master Status is defined in a TxPDO of the CANopen Master as shown in Table 138.

Table 138: CANopen Master object 0x1A84: TxPDO Mapping for CANopen Master Status

SubIndex	Description	Data Type	Value
1	Reserved	UINT32	0x0000000B
2	SYNC Toggle	UINT32	0xF1010C01
3	Device Diag	UINT32	0xF1010D01
4	Align	UINT32	0x00000001
5	PDO Toggle	UINT32	0xF1010F01
6	PDO state	UINT32	0xF1011001
7	Cycle Counter	UINT32	0xF1011110
8	Slave Status Counter	UINT32	0xF1011208
9	Align	UINT32	0x00000008
10	Cycle Time	UINT32	0xF1011410
11	Bus Off	UINT32	0xF1080101
12	Warning Limit reached	UINT32	0xF1080201
13	RX Overflow	UINT32	0xF1080301
14	Align	UINT32	0x00000001
15	TX Overflow	UINT32	0xF1080501
16	Ack Error	UINT32	0xF1080601
17	Align	UINT32	0x0000000A
18-n	Vendor Specific Status	UINT32	
n+1	RX Error Counter	UINT32	0xF1082108
n+2	TX Error Counter	UINT32	0xF1082208

The CANopen Master Status is optional for CANopen Masters and will exclude the CAN Status (0x1A85) in the TxPDO assign.

G.3.8 Object 0x1A85: TxPDO Mapping of the CANopen Status

The CAN Status is defined in a TxPDO of the CANopen Master as shown in Table 139.

Table 139: CANopen Master object 0x1A85: TxPDO Mapping for CAN Status

SubIndex	Description	Data Type	Value
1	Bus Off	UINT32	0xF1080101
2	Warning Limit reached	UINT32	0xF1080201
3	RX Overflow	UINT32	0xF1080301
4	Align	UINT32	0x00000001
5	TX Overflow	UINT32	0xF1080501
6	Ack Error	UINT32	0xF1080601
7	Align	UINT32	0x0000000A
8-n	Vendor Specific Status	UINT32	
n+1	RX Error Counter	UINT32	0xF1082108
n+2	TX Error Counter	UINT32	0xF1082208

The PDO State is mandatory for CANopen Masters.

G.3.9 Object 0x1C12: RxPDO Assign

The RxPDOs of the CANopen Slaves will be assigned in their order in the RxPDO Assign Object 0x1C12 (see Table 140) followed by optional vendor specific RxPDOs.

Table 140: CANopen Master Object 0x1C12: RxPDO Assign

Subindex	Assigned RxPDO
1	RxPDO Mapping Index related to the 1st CANopen Slave with CANopen RxPDOs
2	RxPDO Mapping Index related to the 2nd CANopen Slave with CANopen RxPDOs
...	
n	RxPDO Mapping Index related to the last CANopen Slave with CANopen RxPDOs
n+1	Vendor specific RxPDO if supported
n+2	...
...	

If a CANopen Slave does not have an RxPDO there is no gap in the RxPDO Assign Object.

G.3.10 Object 0x1C13: TxPDO Assign

The TxPDOs of the CANopen Slaves will be assigned in their order in the TxPDO Assign Object 0x1C13 (see Table 141) followed by the TxPDO for PDO State and optionally the TxPDO(s) for the TxPDO Toggle and vendor specific TxPDOs.

Table 141: CANopen Master Object 0x1C13: TxPDO Assign

Subindex	Assigned RxPDO
1	TxPDO Mapping Index related to the 1st CANopen Slave with CANopen TxPDOs
2	TxPDO Mapping Index related to the 2nd CANopen Slave with CANopen TxPDOs
...	
n	TxPDO Mapping Index related to the last CANopen Slave with CANopen TxPDOs
n+1	0x1A81 (TxPDO with PDO States)
n+2	0x1A83 or 0x1A84 or 0x1A85
n+3	0x1A88 (if TxPDO Toggle is supported) or vendor specific TxPDO (if TxPDO Toggle is not supported)
...	

If a CANopen Slave does not have a TxPDO there is no gap in the TxPDO Assign Object.

G.4 EtherCAT State Machine

Table 142 shows the behavior of the CANOpen Master in the different EtherCAT states.

Table 142: CANOpen Master EtherCAT state behavior

EtherCAT State	Behavior / Allowed Operations
INIT	Not connected to the CAN-Bus, CAN-Controller in Reset If the backup parameter were stored, loading the stored configuration from non-volatile memory
PRE-OP	Connect to the CAN-Bus (Start CAN-Controller), when the Device Configuration Data object (0xF800) was downloaded completely, a stored configuration was loaded from non-volatile memory or the Detect Modules command was executed Create the objects 0x6nn0, 0x7nn0, 0x8nn0 when the objects 0x8nn0 and 0x8nn4-0x8nn9 were downloaded completely or a stored configuration was loaded from non-volatile memory Optional: download of PDO Mapping Optional: download of PDO Assign Optional: detect the connected fieldbus slaves and fill the information data objects 0x9nn0) if Detect Modules command is executed Optional: SDO access to the configured or detected CANOpen Slaves via the AoE services
SAFE-OP	CANOpen Boot-Up with the configured CANOpen is started Sending SDO Init Cmds Guarding/Heartbeat is started when the CANOpen Slave is started Sending CANOpen RxPDOs and Receiving CANOpen TxPDOs depending on the Transmission Type The output data of the RxPDOs is in the safe state The input data will be updated with the received TxPDOs, PDO State and TxPDO Toggle Optional: SDO access to the configured or detected CANOpen Slaves via the AoE services
OP	Guarding/Heartbeat is started when the CANOpen Slave is started Sending CANOpen RxPDOs and Receiving CANOpen TxPDOs depending on the Transmission Type The output data of the RxPDOs has process values as received via EtherCAT The input data will be updated with the received TxPDOs, PDO State and TxPDO Toggle Optional: SDO access to the configured or detected CANOpen Slaves via the AoE services

G.5 Device configuration

The Configuration Data objects (0x8000...0x8FFF and 0xF800) can be created in a CANOpen Master device in three different ways:

1. The objects should be created when they are downloaded completely the first time (with CompleteAccess=TRUE), starting with object 0xF800, followed by 0x8000, 0x8004-0x8009, 0x8010, etc.
2. The objects should be created automatically by scanning the CANOpen bus
3. The objects should be created when they were stored as backup parameter

The CANOpen Master has to support at least one of the first two methods.

When supporting the first method the CANOpen Master should support backup parameters because

1. the number of objects to be downloaded in PRE-OP delays the EtherCAT Start-Up that it will only be done if something was changed.
2. the setting of the complete CANOpen configuration is very complex and could be done from a vendor specific tool via the mailbox gateway.

G.6 Acyclic Access to the CANOpen Slaves

The CANOpen Master should support the AoE services to have SDO access to the configured CANOpen slaves. Table 143 shows the relation of AoE parameter.

Table 143: CANopen Master Relation of AoE parameter

AoE Parameter		CANopen Relation
Port		1000 + Node Address of the CANopen Slave
Index Group, Bit 16-31 = 0		SDO Service
	Command	2 = SDO Upload, 3 = SDO Download
	Index Group, Bit 0-15	Index
	Index Offset, Bit 0-7	Subindex
	Index Offset, Bit 8-15	Response Timeout (optional) for actual request in 100 ms (0 means configured response timeout (-> 0x8nn0:23h))
	Index Offset, Bit 16-31	Reserved
Index Group, Bit 16-31 = 1		SDO Block Service
	Command	2 = SDO Upload, 3 = SDO Download
	Index Group, Bit 0-7	Index
	Index Offset, Bit 0-7	Subindex
	Index Offset, Bit 8-15	Response Timeout (optional) for actual request in 100 ms (0 means configured response timeout (-> 0x8nn0:23h))
	Index Offset, Bit 16-31	Reserved
Index Group, Bit 16-31 = 2-0xFFFFF		Vendor_Specific

G.7 Synchronization

The CANopen Master shall support at the least one the synchronization modes as defined in [6]. This chapter describes the synchronization modes applied to the CANopen Master.

G.7.1 FreeRun Mode

In the FreeRun Mode the EtherCAT cycle is not synchronized with the CANopen Master's cycle.

G.7.2 Synchron Mode

In the Synchron Mode the EtherCAT cycle is synchronized with the CANopen Master's cycle. If Sync CAN PDOs are defined, the SYNC message shall be sent at the beginning of the CANopen Master's cycle. The Sync Cycle Time shall be equal or a multiple of the EtherCAT Cycle time.

G.7.3 DC Mode

In DC Mode the EtherCAT cycle is synchronized with the CANopen Master's cycle via Distributed Clocks. If Sync CAN PDOs are defined, the SYNC message shall be sent at the beginning of the CANopen Master's cycle. The Sync Cycle Time shall be equal or a multiple of the EtherCAT Cycle time.

Annex H CANopen Slave (Module Profile Number 5110)

H.1 General

The CANopen Slave gateway is based on the modular device profile (fieldbus gateway). The CANopen Slave gateway supports 1 module (CANopen Slaves).

H.2 Object Dictionary of the CANopen Slave Device

The object dictionary of the device contains the following object areas shown in Table 144.

Table 144: CANopen Slave device Object Dictionary Structure

Index	Object Dictionary Area
0x0000...0x0FFF	Data Type Area
0x1000...0x1FFF	Communication Area
0x6000...0x6FFF	Input Area (RxPDOs of the CANopen slave)
0x7000...0x7FFF	Output Area (TxPDOs of the CANopen slave)
0x8000...0x8FFF	Configuration Area (configuration of the CANopen slave)
0xF000...0xFFFF	Device Area

H.2.1 Object Areas of the CANopen Slave (Module)

H.2.1.1 Output Data

The Output Data represent the CAN TxPDOs of the CANopen Slave. The Output Data is mandatory for a CANopen Slave with CAN TxPDOs.

Table 145: CANopen Slave Objects 0x7nn0...0x7nn1: Output Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x7000	1	TxPDO 001	OCTET-STRING	RO	Represents TxPDO 1 of the CANopen Slave
	2	TxPDO 002	OCTET-STRING	RO	Represents TxPDO 2 of the CANopen Slave
..					
0x7001	1	TxPDO 256	OCTET-STRING	RO	Represents TxPDO 256 of the CANopen Slave
..					
	255	TxPDO 510	OCTET-STRING	RO	Represents TxPDO 510 of the CANopen Slave
...					

If a TxPDO does not exist, the related entry does not exist too and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

When no entry of an object is defined (e.g. object 0x7001 does not exist if the related CANopen Slave does not support the TxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

Depending on the Transmission type of the CANopen Slave TxPDOs the CANopen Slave will send the TxPDO CAN messages when the Output Data was written by the EtherCAT master.

The objects 0x7nn2-0x7nn3, 0x7nn6-0x7nn7, 0x7nnA-0x7nnB, 0x7xxE-0x7nnF are vendor specific but shall be used according to the Modular Device Profile. The objects 0x7nn4-0x7nn5, 0x7nn8-0x7nn9 and 0x7nnC-0x7nnD are reserved for profile extensions.

H.2.1.2 Input Data

The Input Data of the CANopen Slave represent the RxPDOs of the CANopen Slave. The Input Data is mandatory for a CANopen Slave with CAN RxPDOs.

Table 146: CANopen Slave Objects 0x6nn0...0x6nn1: Input Data

Index	Subindex	Name	Data Type	Access	Description / Value
0x6000	1	RxPDO 001	OCTET-STRING	RO	Represents RxPDO 1 of the CANopen Slave
	2	RxPDO 002	OCTET-STRING	RO	Represents RxPDO 2 of the CANopen Slave
..					
0x6001	1	RxPDO 256	OCTET-STRING	RO	Represents RxPDO 256 of the CANopen Slave
..					
	255	RxPDO 510	OCTET-STRING	RO	Represents RxPDO 510 of the CANopen Slave
...					

If a RxPDO does not exist, the related entry does not exist too and a SDO Upload or a SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex doesn't exist).

In the SDO Upload response with CompleteAccess = TRUE the not existing entries shall have no space.

When no entry of an object is defined (e.g. object 0x6001 does not exist if the CANopen Slave does not support the RxPDOs 256 to 510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The received RxPDOs of the CANopen slave will be copied in the Input Data which is read by the EtherCAT master.

H.2.1.3 Configuration Data

The Configuration Data contain the CANopen communication parameter the CANopen Slave needs to establish a CANopen connection to the CANopen Master. The Configuration Data is mandatory for a CANopen Slave.

CAN Communication Parameter

Table 147: CANopen Slave Object 0x8000: CAN Communication Parameter

Index	Subindex	Name	Data Type	Access	Description / Value
0x8000	1	Node Address	UINT16	RW	Node Address of the CANopen Slave (0=Node Address will be generated locally (e.g. by an address switch, 1-126: Node ID of the CANopen slave)
	4	Device Type	UINT32	RW	Device Type (Object 0x1000) of the CANopen Slave
	5	Vendor ID	UINT32	RW	Vendor ID (Entry 0x1018:01) of the CANopen Slave
	6	Product Code	UINT32	RW	Product Code (Entry 0x1018:02) of the CANopen Slave
	7	Revision	UINT32	RW	Revision (Entry 0x1018:03) of the CANopen Slave
	8	Serial Number	UINT32	RW	Serial Number (Entry 0x1018:04) of the first CANopen Slave
	29	Network Flags	UINT16	RW	Network Flags to route vendor specific services over CANopen, shall be 0, if not used
	30	Network Port	UINT16	RW	Network Port to route vendor specific services over CANopen, shall be 0, if not used
	31	Network Segment Address	OCTET-STRING[6]	RW	Network Segment Address to route vendor specific services over CANopen, shall be 0, if not used
	32		UINT16	RW	Reserved for module of CANopen Master, shall be 0
	33	Guarding Time	UINT16	RW	Default Guarding Time (Object 0x100C) set to the CANopen Slave, 0: no guarding
	34	Life Time Factor	UINT16	RW	Default Life Time Factor (Object 0x100D) set to the CANopen Slave, 0: no guarding
	35		UINT16	RW	Reserved for module of CANopen Master, shall be 0
	36		UINT16	RW	Reserved for module of CANopen Master, shall be 0
	37-n			RW	32 bytes of vendor specific configuration data, shall be zero, when accessing with CompleteAccess=TRUE, if not used
...		...			

The object 0x8000 is mandatory for a CANopen Slave.

CAN TxPDO Configuration

Table 148: CANopen Slave Object 0x8006...0x8007: TxPDO Configuration

Index	Subindex	Name	Data Type	Access	Description / Value
0x8006	1	TxPDO 001	OCTET-STRING[12]	RW	TxPDO 1 Configuration of the CANopen Slave
					Byte 0-3: Default COB-ID of the TxPDO (Entry 0x1800:01 of the CANopen Slave)
					Byte 4: Default Transmission Type of the TxPDO (Entry 0x1800:02 of the CANopen Slave)
					Byte 5: Length of the TxPDO (1-8)
					Byte 6-7: Default Inhibit Time of the TxPDO (Entry 0x1800:03 of the CANopen Slave)
					Byte 8-9: Default Event Time of the TxPDO (Entry 0x1800:05 of the CANopen Slave)
					Byte 10-11: Flags Bit 0-7: reserved for future use Bit 8-15: vendor specific
	2	TxPDO 002	OCTET-STRING[12]	RW	TxPDO 2 Configuration of the CANopen Slave
	...				
0x8007	1	TxPDO 256	OCTET-STRING[12]	RW	TxPDO 256 Configuration of the CANopen Slave
	...				
	255	TxPDO 510	OCTET-STRING[12]	RW	TxPDO 510 Configuration of the CANopen Slave
	...				

If a TxPDO does not exist, the related entry exists as NULL-Entry if entries of this object follows. A SDO Upload or SDO Download service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing). A SDO Information service with the related subindex addressed shall return the entry description with DataType=0 and BitLength=80 (see chapter SDO Complete Access in [6]). The definition of NULL-Entries is needed that a SDO Download with CompleteAccess=TRUE can be supported.

If a TxPDO does not exist, the related Subindex does not exist if no entries of this object follows. A SDO Upload, SDO Download or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x8007 does not exist if the CANopen Slave does not support the TxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The object 0x8006 is mandatory for a CANopen Slave. The object 0x8007 is optional for a CANopen Slave.

CAN RxPDO Configuration

Table 149: CANopen Slave Object 0x8008...0x8009: RxPDO Configuration

Index	Subindex	Name	Data Type	Access	Description / Value
0x8008	1	RxPDO 001	OCTET-STRING[12]	RW	RxPDO 1 Configuration of the CANopen Slave
					Byte 0-3: Default COB-ID of the RxPDO
					Byte 4: Default Transmission Type of the RxPDO
					Byte 5: Length of the RxPDO (1-8)
					Byte 6-7: Default Inhibit Time of the RxPDO
					Byte 8-9: Default Event Time of the RxPDO
					Byte 10-11: Flags Bit 0-7: Reserved for future use Bit 8-15: Vendor specific
	2	RxPDO 002	OCTET-STRING[12]	RW	RxPDO 2 Configuration of the CANopen Slave
	...				
0x8009	1	RxPDO 256	OCTET-STRING[12]	RW	RxPDO 256 Configuration of the CANopen Slave
	...				
	255	RxPDO 510	OCTET-STRING[12]	RW	RxPDO 510 Configuration of the CANopen Slave

If a RxPDO does not exist, the related entry exists as NULL-Entry if entries of this object follows. A SDO Upload or SDO Download service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing). A SDO Information service with the related subindex addressed shall return an entry description with DataType=0 and BitLength=80.

If a RxPDO does not exist, the related does not exist if no entries of this object follows. A SDO Upload, SDO Download or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex not existing).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x8009 does not exist if the CANopen Slave does not support the RxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object not existing).

The object 0x8008 is mandatory for a CANopen Slave. The object 0x8009 is optional for a CANopen Slave.

CAN TxPDO Mapping

Table 150: CANopen Slave Object 0x800A-0x800B: TxPDO Mapping

Index	Subindex	Name	Data Type	Access	Description / Value
0x800A	1	TxPDO 001	OCTET-STRING[n]	RO	TxPDO 1 Mapping of the CANopen Slave
					Byte 0-3: 1st mapping entry of the TxPDO (Entry 0x1A00:01 of the CANopen Slave)
					Byte 4-7: 2nd mapping entry of the TxPDO (Entry 0x1A00:02 of the CANopen Slave)
					...
	2	TxPDO 002	OCTET-STRING[n]	RO	TxPDO 2 Mapping of the CANopen Slave
	...				
0x800B	1	TxPDO 256	OCTET-STRING[n]	RO	TxPDO 256 Mapping of the CANopen Slave found
	...				
	255	TxPDO 510	OCTET-STRING[n]	RO	TxPDO 510 Mapping of the CANopen Slave

If a TxPDO does not exist, the related entry does not exist. A SDO Upload or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex doesn't exist).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x800B does not exist if the related CANopen Slave does not support the TxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object doesn't exist).

The objects 0x800A-0x800B are optional for a CANopen Slave.

CAN RxPDO Mapping

Table 151: CANopen Slave Object 0x800C-0x800D: RxPDO Mapping

Index	Subindex	Name	Data Type	Access	Description / Value
0x800C	1	RxPDO 001	OCTET-STRING[n]	RO	RxPDO 1 Mapping of the CANopen Slave
					Byte 0-3: 1st mapping entry of the RxPDO (Entry 0x1600:01 of the CANopen Slave)
					Byte 4-7: 2nd mapping entry of the RxPDO (Entry 0x1600:02 of the CANopen Slave)
					...
	2	RxPDO 002	OCTET-STRING[n]	RO	RxPDO 2 Mapping of the CANopen Slave
	...				
0x800D	1	RxPDO 256	OCTET-STRING[n]	RO	RxPDO 256 Mapping of the CANopen Slave
	...				
	255	RxPDO 510	OCTET-STRING[n]	RO	RxPDO 510 Mapping of the CANopen Slave
...					

If a RxPDO does not exist, the related entry does not exist. A SDO Upload or SDO Information service with the related subindex addressed shall return the abort code 0x06090011 (Subindex doesn't exist).

When accessing the object with CompleteAccess=TRUE the values of the NULL-entries shall be zero. The values from Subindex 1 until the last Subindex (including NULL-Entries) shall be packed one after another.

When no entry of an object is defined (e.g. object 0x900E does not exist if the CANopen Slave does not support the RxPDOs 256-510), the object does not exist and a SDO Upload or a SDO Information service with the related index addressed shall return the abort code 0x06020000 (object doesn't exist).

The objects 0x800C...0x800D are optional for a CANopen Slave.

The objects 0x8001...0x8005 are vendor specific but shall be used according to the Modular Device Profile. The objects 0x800E...0x800F are reserved for profile extensions.

H.2.1.4 Diagnosis Data

No standardized Diagnosis Data defined.

H.2.2 Object Areas of the Device

The following objects of the Device Area (see Table 152) are standardized for a CANopen Slave. All other objects in the area 0xF000...0xFFFF are reserved for profile extensions.

Table 152: CANopen Slave Device Object Areas

Index	Object Name
0xF000	Modular device profile
0xF100	CANopen Status
0xF108	CAN Status
0xF800	CAN Bus parameter set

H.2.2.1 Object 0xF000: Modular Device Profile

The Modular device profile object (0xF000) shown in Table 153 is mandatory for a CANopen Slave.

Table 153: CANopen Slave Object 0xF000: Modular Device Profile

Index	Subindex	Name	Data Type	Access	Description / Value
0xF000	1	Index distance	UINT16	RO	Index distance between two modules = 0x10
	2	Maximum number of modules	UINT16	RO	Maximum number of modules = 1
	3	General configuration	UINT32	RO	Available entries in objects 0x8nn0 = 0x0000007D

H.2.2.2 Object 0xF100: CANopen Status

The CANopen status object (0xF100) shown in Table 154 is mandatory for a CANopen Slave.

Table 154: CANopen Slave Object 0xF100: CANopen Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF100	1	CANopen Communication Status	ENUM[8]	RO	Status of the CANopen connection to the CANopen Master
					2: Guarding error
					22: Event Timer error,
					127: Baudrate search
					128: CANopen Slave is OPERATIONAL but not all RxPDOs were received
					129: CANopen Slave is PRE-OPERATIONAL
					130: CANopen Slave is STOPPED
					0: CANopen Slave is OPERATIONAL and all RxPDOs were received
	9		BIT7		Reserved for future use
	16	TxPDO State	BOOL	RO	Same value as 0x1800:07, will be set, if the Input Data is not valid (0xF100:01 > 0)

H.2.2.3 Object 0xF108: CAN Status

The CAN Status object (0xF108) as shown in Table 155 is optional for a CANopen Slave.

Table 155: CANopen Slave Object 0xF108: CAN Status

Index	Subindex	Name	Data Type	Access	Description / Value
0xF108	1	Bus Off	BOOLEAN	RO	TRUE: CAN-Controller is bus off
	2	Warning Limit reached	BOOLEAN	RO	TRUE: Warning Limit of the CAN-Controller is reached
...	3	RX Overflow	BOOLEAN	RO	TRUE: CAN-Controller has a RX message overflow
	4		BIT1	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=1)
	5	TX Overflow	BOOLEAN	RO	TRUE: TX message overflow
	6	Ack Error	BOOLEAN	RO	TRUE: CAN-Controller could not sent a message
	7		BIT2	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=2)
	9		BIT8	RO	NULL:Entry: Reserved for future use, shall be zero (Data-Type=0, Bit-Length=7)
	17-32			RO	Reserved for 16 bits of vendor specific status, the Subindex shall be the bit position as for Subindex 1-16
	33	Rx Error Counter	UINT8	RO	RX Error Counter from CAN-Controller
	34	Tx Error Counter	UINT8	RO	TX Error Counter from CAN-Controller

H.2.2.4 Object 0xF800: CAN Bus parameter set

The CAN Bus parameter set object (0xF800) as shown in Table 156 is mandatory for a CANopen Slave.

Table 156: CANopen Slave Object 0xF800: CAN Bus Parameter Set

Index	Subindex	Name	Data Type	Access	Description / Value
0xF800	1	CANopen Node Address, CANopen Slave	UINT8	RW	CANopen Node Address of the CANopen Slave, same value as 0x8000:01
	2	Baudrate	UINT8	RW	Baudrate (0 = 1 MBaud, 1 = 800 kBaud, 2 = 500 kBaud, 3 = 250 kBaud, 4 = 125 kBaud, 5 = 100 kBaud, 6 = 50 kBaud, 7 = 20 kBaud, 8 = 10 kBaud, 255 = bus timing corresponding to bus timing registers)
	3		UINT16	RW	reserved for CANopen master, shall be 0x80
	4		UINT32	RW	reserved for CANopen master, shall be 0
	5	Bustiming Register	UINT32	RW	Bit 0-7: BT0-Register Bit 8-15: BT1-Register Bit 16-23: BT2-Register Bit 24-31: BT3-Register
	6-n			RW	32 bytes of vendor specific configuration data, shall be zero, when accessing with CompleteAccess=TRUE, if not used

H.3 PDO Mapping and Assign

H.3.1 Object 0x1600: RxPDO Mapping of the CANopen Slave

For the CANopen Slave one RxPDO Mapping object is reserved as shown in Table 157.

Table 157: CANopen Slave Object 0x1600: RxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1600	1	Subindex 001	UINT32	0x7000ppss (pp depends on the related CAN TxPDO, ss is the size of this CAN TxPDO in bits, starting with the first CAN TxPDO of the CANopen Slave)
	2	Subindex 002	UINT32	0x7000ppss (pp depends on the related CAN TxPDO, ss is the size of this CAN TxPDO in bits, starting with the first CAN TxPDO of the CANopen Slave)
	...			

If the CANopen Slave has no CAN TxPDOs the related CoE RxPDO Mapping object does not exist.

The CoE RxPDO Mapping is mandatory for the CANopen Slave.

H.3.2 Object 0x1A00: TxPDO Mapping of the CANopen Slave

For the CANopen Slave one TxPDO Mapping object is reserved as shown in Table 158.

Table 158: CANopen Slave Object 0x1A00: TxPDO Mapping

Index	Subindex	Name	Data Type	Description / Value
0x1A00	1	Subindex 001	UINT32	0x6000ppss (pp depends on the related CAN RxPDO, ss is the size of this CAN RxPDO in bits, starting with the first CAN RxPDO of the CANopen Slave)
	2	Subindex 002	UINT32	0x6000ppss (pp depends on the related CAN RxPDO, ss is the size of this CAN RxPDO in bits, starting with the first CAN RxPDO of the CANopen Slave)
	...			

If the CANopen Slave has no CAN TxPDOs the related CoE TxPDO Mapping object does not exist.

The CoE TxPDO Mapping is mandatory for the CANopen Slave.

H.3.3 Object 0x1A83: TxPDO Mapping of the CANopen Status

The CANopen Status is defined in a TxPDO of the CANopen Slave as shown in Table 159.

Table 159: CANopen Slave Object 0x1A83: TxPDO Mapping of CANopen Status

Subindex	Name	Data Type	Value
1	CANopen Communication Status	UINT32	0xF1000108
2	Align	UINT32	0x00000007
3	TxPDO State	UINT32	0x18000701

H.3.4 Object 0x1A85: TxPDO Mapping of the CAN Status

The CAN Status is defined in a TxPDO of the CANopen Slave as shown in Table 160.

Table 160: CANopen Slave Object 0x1A85: TxPDO Mapping of CAN Status

Subindex	Name	Data Type	Value
1	Bus Off	UINT32	0xF1080101
2	Warning Limit reached	UINT32	0xF1080201
3	RX Overflow	UINT32	0xF1080301
4	Align	UINT32	0x00000001
5	TX Overflow	UINT32	0xF1080501
6	Ack Error	UINT32	0xF1080601
7	Align	UINT32	0x0000000A
8-n	Vendor Specific Status	UINT32	
n+1	RX Error Counter	UINT32	0xF1082108
n+2	TX Error Counter	UINT32	0xF1082208

The CAN Status is mandatory for CANopen Slaves.

H.3.5 Object 0x1C12: RxPDO Assign

The CoE RxPDO of the CANopen Slave will be assigned first in the RxPDO Assign Object 0x1C12 (see Table 161) followed by optional vendor specific RxPDOs.

Table 161: CANopen Slave Object 0x1C12: RxPDO Assign

Subindex	Assigned RxPDO
1	0x1600
2	Vendor specific RxPDO if supported
...	

If the CANopen Slave does not have a RxPDO, then there is no gap in the RxPDO Assign Object.

H.3.6 Object 0x1C13: TxPDO Assign

The CoE TxPDO of the CANopen Slave will be assigned first in the TxPDO Assign Object 0x1C13 (see Table 162) followed by the TxPDO for the CANopen Status, the TxPDO for the CAN Status and vendor specific TxPDOs.

Table 162: CANopen Slave Object 0x1C13: TxPDO Assign

Subindex	Assigned TxPDO
1	0x1A00
2	0x1A83
3	0x1A85
4	Vendor specific TxPDO if supported
...	

If a CANopen Slave does not have a TxPDO, then there is no gap in the TxPDO Assign Object.

H.4 EtherCAT State Machine

Table 163 shows the behavior of the CANopen Slave in the different EtherCAT states.

Table 163: CANopen slave EtherCAT state behavior

EtherCAT State	Behavior / Allowed Operations
INIT	Not connected to the CAN-Bus, CAN-Controller in Reset If the backup parameter were stored, loading the stored configuration from non-volatile memory
PRE-OP	Download the Device Configuration Data object (0xF800) completely or load a stored configuration was loaded from non-volatile memory Create the objects 0x6nn0, 0x7nn0, 0x8nn0 when the objects 0x8nn0 and 0x8nn4-0x8nnD were downloaded completely or a stored configuration was loaded from non-volatile memory Optional: download of PDO Mapping Optional: download of PDO Assign
SAFE-OP	Connect to the CAN bus (start the CAN-controller) and wait for the Boot-Up from the CANopen master Guarding/Heartbeat is started when the CANopen Slave is started by the CANopen Master Sending CAN TxPDOs and Receive CAN RxPDOs depending on the Transmission Type when the CANopen Slave is started by the CANopen Master The output data of the CAN TxPDOs is in the safe state The input data will be updated with the received CAN RxPDOs, CANopen Status and CAN Status
OP	Guarding/Heartbeat is started when the CANopen Slave is started by the CANopen Master Sending CAN TxPDOs and Receive CAN RxPDOs depending on the Transmission Type when the CANopen Slave is started by the CANopen Master The output data of the CAN TxPDOs has process values as received via EtherCAT The input data will be updated with the received CAN RxPDOs, CANopen Status and CAN Status

H.5 Device configuration

The Configuration Data objects (0x8000...0x8FFF and 0xF800) can be created in a CANopen Slave device in two different ways:

1. The objects should be created when they are downloaded completely the first time (with CompleteAccess=TRUE), starting with object 0xF800, followed by 0x8000, 0x8004-0x800D.
2. These objects should be created when they were stored as backup parameter before

The CANopen Slave shall support at least one of the two methods.

H.6 Synchronization

The CANopen Slave shall support the Free Run mode as defined in [6].

H.6.1 FreeRun Mode

In the FreeRun Mode the EtherCAT cycle is not synchronized with the CANopen Slave's cycle.

Annex I DeviceNet Master (Module Profile Number 5200)

To be defined.

Annex J DeviceNet Slave (Module Profile Number 5210)

To be defined

Annex K IO-Link Master (Module Profile Number 6220)

The IO-Link master profile is specified in the document ETG.5001.6220 (www.ethercat.org/ETG5001).